



Universidad
Francisco de
Vitoria

UFV Madrid

UNIVERSIDAD FRANCISCO DE VITORIA

ESCUELA POLITÉCNICA SUPERIOR

GRADO EN INGENIERÍA MATEMÁTICA

PROYECTO FINAL DE GRADO

MODALIDAD INGENIERÍA

Asistente virtual para el estudio de gramáticas regulares e independientes de contexto y autómatas finitos.

Ignacio de Lecea Jiménez
Convocatoria de Junio 2025

CALIFICACIÓN DEL PROYECTO FINAL DE GRADO

CUALITATIVA:	
NUMÉRICA:	

Conforme Presidente:	Conforme Secretario:
Fdo.:	Fdo.:

Conforme Vocal:	Conforme Vocal:	Conforme Vocal:
Fdo.:	Fdo.:	Fdo.:

Lugar y fecha: Pozuelo de Alarcón, a ____ de _____ de 202__

Agradecimientos

Especial agradecimiento a Juan José Escribano, que no solo fue el precursor para la realización de este proyecto, sino que me ha acompañado a lo largo de todo el camino, siendo su implicación crucial para lograr un desarrollo funcional y óptimo del Asistente virtual, tanto en la recopilación de la documentación como en la evaluación humana del *Copilot* y asesoramiento en diversos impedimentos.

Elena Gutiérrez Cruz por su asesoramiento en el uso de un sistema basado en *RAG* y todos sus componentes cruciales, especialmente la base de datos vectorial, y en la elección del formato *Markdown* para el desarrollo de la base de conocimiento.

Moises Martínez por su asesoramiento en la resolución de impedimentos técnicos, desde la tentativa original del desarrollo en java hasta su final despliegue en *HugginFace Spaces*.

Resumen

La Inteligencia Artificial, en constante evolución, ofrece nuevas oportunidades para transformar la educación. Este proyecto propone el desarrollo de Fluxo, una aplicación diseñada como asistente virtual para apoyar a docentes y estudiantes en el estudio de gramáticas regulares, gramáticas independientes de contexto y autómatas finitos.

Fluxo se distingue por integrar dos funcionalidades principales:

1. *Chatbot* especializado: Diseñado para facilitar la consulta de documentación técnica, este módulo permite a los usuarios obtener respuestas instantáneas y precisas sobre conceptos teóricos y prácticos relacionados con gramáticas y autómatas.
2. *Copilot* educativo: Este módulo actúa como un asistente virtual, proporcionando retroalimentación detallada y granulada sobre las soluciones planteadas por los estudiantes. Identifica errores, sugiere mejoras y guía al usuario hacia una comprensión más profunda del problema.

La solución final consistió en una aplicación web accesible, desarrollada con *Streamlit*, que utilizó el modelo *Llama 4 Maverick* a través de *GroqCloud*, alimentada con una base de datos vectorial en *Qdrant* y desplegada en *Hugging Face Spaces* para garantizar acceso multiplataforma y sin necesidad de instalación.

Palabras claves

Computabilidad, lenguajes formales, autómatas finitos, gramáticas formales, lenguajes regulares, lenguajes independientes de contexto, máquinas de Turing.

Abstract

Artificial Intelligence, in constant evolution, offers new opportunities to transform education. This project proposes the development of Fluxo, an application designed as a virtual assistant to support teachers and students in the study of regular grammars, context-free grammars, and finite automata.

Fluxo is distinguished by integrating two main functionalities:

- 1. Specialized chatbot: Designed to facilitate the consultation of technical documentation, this module allows users to obtain instant and precise answers about theoretical and practical concepts related to grammars and automata.*
- 2. Educational copilot: This module acts as a virtual assistant, providing detailed and granular feedback on the solutions proposed by students. Fluxo identifies errors, suggests improvements, and guides the user toward a deeper understanding of the problem.*

The final solution involved an accessible web application developed with Streamlit, leveraging the Llama 4 Maverick model through GroqCloud, powered by a Qdrant vector database, and deployed on Hugging Face Spaces to ensure multiplatform access without installation requirements.

Keywords

Computability, formal languages, finite automata, formal grammars, regular languages, context-free languages, Turing machines.

Índice de Contenidos

1. Introducción	1
1.1. Contexto	2
2. Investigación previa	3
2.1. Precedentes	3
2.2. Tecnologías aplicadas.....	5
2.3. Valor diferencial del proyecto	7
3. Objetivos	9
3.1. Objetivo general	9
3.2. Objetivos específicos	9
3.3. Métodos de Validación.....	10
4. Plan de Desarrollo del Proyecto	13
4.1. Metodología.....	13
4.2. Tecnologías.....	14
4.3. Plan de desarrollo del proyecto	15
4.3.1. Análisis y definición del problema.....	16
4.3.2. Desarrollo del modelo matemático.....	17
4.3.3. Implementación y análisis de resultados	20
4.4. Plan de Trabajo	21
4.5. Recursos.	22
4.6. Costes	23
5. Desarrollo de la Solución Técnica	25
5.1. PT1 Investigación y recopilación de información	25
5.1.1. Estudio detallado de lenguajes formales, gramáticas y autómatas	25
5.1.2. Análisis de herramientas existentes.....	26
5.1.3. Investigación sobre sistemas RAG y bases de datos vectoriales.....	26
5.1.4. Evaluación comparativa de tecnologías y <i>frameworks</i>	28
5.1.5. Resultados obtenidos	28

5.2.	PT2 Diseño conceptual del sistema.....	29
5.2.1.	Diseño del sistema RAG para gestión del conocimiento.....	29
5.2.2.	Definición del formato estructurado para la base de conocimiento.....	30
5.2.3.	Conceptualización de los módulos educativos	30
5.2.4.	Diseño preliminar de la interfaz de usuario	31
5.2.5.	Resultados y desafíos del paquete	31
5.3.	PT3 Desarrollo técnico.....	31
5.3.1.	Creación y estructuración de la base de conocimiento en <i>Markdown</i>	32
5.3.2.	Conversión de los archivos <i>Markdown</i> a <i>embeddings</i> vectoriales y almacenamiento en <i>Qdrant</i>	32
5.3.3.	Implementación del sistema RAG integrando recuperación semántica y generación de respuestas.....	34
5.3.4.	Desarrollo de la aplicación web con <i>Streamlit</i> y despliegue en <i>Hugging Face Spaces</i>	36
5.4.	PT4 pruebas y validación	38
5.5.	PT5 documentación final	43
5.5.1.	Estructura y contenidos de la memoria final	43
5.5.2.	Proceso de elaboración	44
6.	Resultados.....	45
7.	Implicaciones Éticas e Impacto Social.....	47
7.1.	Marco ético y legal.....	48
7.2.	Valor del proyecto.....	48
7.3.	Alcance.....	49
7.4.	Responsabilidad e impacto.....	49
7.5.	Conclusiones	51
8.	Conclusiones.....	53
9.	Bibliografía.....	55
Anexo A.		57

Índice de Tablas

<i>Tabla 1: Componentes Mean Pooling.....</i>	<i>18</i>
<i>Tabla 2: Resultados.....</i>	<i>40</i>
<i>Tabla 3: Benchmark.....</i>	<i>45</i>
<i>Tabla 4: Registro de Riesgos</i>	<i>50</i>

Índice de Figuras

<i>Figura 1: Comparación Datasets</i>	<i>4</i>
<i>Figura 2: Inteligencia</i>	<i>6</i>
<i>Figura 3: Velocidad</i>	<i>6</i>
<i>Figura 4: Precio.....</i>	<i>6</i>
<i>Figura 5: Funcionalidad de Totalf.....</i>	<i>13</i>
<i>Figura 6: Diagrama de Gantt.</i>	<i>21</i>
<i>Figura 7: Arquitecturas RAG aplicables. [16]</i>	<i>27</i>
<i>Figura 8: Diagrama de flujo.....</i>	<i>29</i>
<i>Figura 9: Diagrama de flujo de conversión MD a Embedding.</i>	<i>32</i>
<i>Figura 10: Funcionalidad de consulta de documentación.</i>	<i>36</i>
<i>Figura 11: Funcionalidad de asistente virtual.....</i>	<i>37</i>
<i>Figura 12: Dashboard Groq.....</i>	<i>41</i>
<i>Figura 13: Dashboard Qdrant.....</i>	<i>42</i>
<i>Figura 14: Versión java.....</i>	<i>45</i>
<i>Figura 15: Funcionalidad de asistente virtual en java.....</i>	<i>46</i>
<i>Figura 16: Matriz de Riesgos.....</i>	<i>50</i>

Lista de Acrónimos

Acrónimo	Significado
LLM	Large Language Model
IA	Inteligencia Artificial
JFLAP	Java Formal Languages and Automata Package
API	Application Programming Interface
RAG	Retrieval-Augmented Generation
JSON	JavaScript Object Notation
LPU	Language Processing Unit
CPU	Central Processing Unit
GPU	Graphical Processing Unit
GIC	Gramática independiente de contexto
GR	Gramática regular
CYK	Cocke-Younger-Kasami
FNC	Forma Normal de Chomsky
FNG	Forma Normal de Greibach
AFD	Autómata Finito Determinista
AFND	Autómata Finito No Determinista
APV	Autómata a Pila por Vaciado
APF	Autómata a Pila por Estados Finales
HNSW	Hierarchical Navigable Small World
KNN	K-Nearest Neighbors
BERT	Bidirectional Encoder Representations from Transformers
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
RAM	Random-access memory
IEEE	Institute of Electrical and Electronics Engineers
APA	American Psychological Association

1. INTRODUCCIÓN

En los últimos años, la inteligencia artificial ha experimentado un avance exponencial, introduciéndose en prácticamente todas las áreas tecnológicas, desde el análisis de datos y la automatización de procesos industriales hasta la optimización de servicios y la personalización de la experiencia de usuario. Estas aplicaciones han permitido mejorar la eficiencia, la precisión y el aprendizaje en múltiples disciplinas, agilizando procesos y eliminando tareas intermedias.

En el ámbito de la computación teórica, la enseñanza de gramáticas formales y autómatas finitos constituye un pilar fundamental. Herramientas como *JFLAP* han facilitado la representación gráfica de conceptos complejos, aunque presentan limitaciones en cuanto a retroalimentación pedagógica y resolución guiada de problemas. El objetivo de este trabajo fue desarrollar *Fluxo*, un asistente virtual dirigido a estudiantes y docentes para el estudio de gramáticas regulares, gramáticas independientes de contexto y autómatas finitos. El proyecto buscaba superar las carencias de herramientas tradicionales mediante la integración de inteligencia artificial generativa, combinando teoría, práctica y tecnología avanzada en una solución accesible y multiplataforma, centrada en ofrecer una experiencia educativa más interactiva, adaptativa y efectiva.

El desarrollo se dividió en tres fases principales:

1. Documentación unificada y problemas resueltos:

Se creó una base de datos de conocimiento, almacenándola en la base de datos vectorial *Qdrant*, que incluyó teoría y ejercicios resueltos sobre temas como lenguajes formales, gramáticas, autómatas finitos, autómatas a pila y máquinas de Turing. Esta documentación sirvió para entrenar el modelo de IA generativa utilizado en el proyecto, permitiendo la recuperación eficiente de información relevante mediante técnicas de *RAG*.

2. Implementación del modelo de IA:

Se empleó el modelo generativo *Llama 4 Maverick*, versión escalable del *LLM* Llama 4, ajustado y refinado para responder a dos necesidades clave:

Chatbot especializado: Proporciona respuestas instantáneas a consultas sobre gramáticas y autómatas.

Copilot educativo: Funciona como asistente virtual, ofreciendo retroalimentación detallada paso a paso en la resolución de problemas. A diferencia de otras herramientas, este módulo identifica errores específicos, sugiere mejoras y guía al usuario hacia una comprensión más profunda del problema, fomentando una experiencia de aprendizaje activa y constructiva.

3. Despliegue multiplataforma y accesible:

Se optó por una solución web basada en *Streamlit*, alojada en *Hugging Face Spaces*, que permitió el acceso desde cualquier dispositivo sin necesidad de instalación, facilitando así su adopción en entornos educativos diversos.

1.1. CONTEXTO

Durante mi formación académica, la Inteligencia Artificial se utilizó como una herramienta esencial de apoyo en la mayoría de las asignaturas de la carrera. Este aprendizaje teórico y práctico se reforzó en las prácticas realizadas en Mapfre, donde participé en dos proyectos de integración de IA en sistemas internos: uno orientado a la gestión y optimización de costes, y otro enfocado al control y automatización de la documentación interna. Estos proyectos me permitieron experimentar directamente el impacto transformador de la IA en los procesos empresariales, lo que intensificó el interés en la materia y motivó la exploración de nuevas aplicaciones tecnológicas.

A nivel educativo, observé cómo la IA modificó radicalmente el sistema de aprendizaje. Muchas habilidades y tareas, como el aprendizaje exhaustivo de la sintaxis en lenguajes de programación, se optimizaron mediante herramientas como *ChatGPT*, que permitieron a los estudiantes centrarse en la lógica y el diseño de soluciones. Con una base sólida, se demostró que era posible desarrollar programas complejos utilizando la IA como soporte, liberando tiempo y esfuerzo para profundizar en conceptos avanzados.

A partir de esta experiencia, surgió la idea de desarrollar un *chatbot* para el portal universitario con el objetivo de agilizar las consultas estudiantiles. Sin embargo, al tratarse de una propuesta limitada en alcance, no fue aceptada como proyecto. Posteriormente, el profesor Juan José Escribano, encargado de la asignatura de autómatas, sugirió trabajar en un nuevo proyecto que identificara áreas de mejora dentro de *JFLAP* y que pudiera beneficiarse de una integración con IA.

2. INVESTIGACIÓN PREVIA

El desarrollo de Fluxo, requirió una investigación exhaustiva que abarcó el contexto teórico, el análisis de herramientas existentes y el estudio de tecnologías aplicadas. Esta sección detalla los antecedentes, hallazgos relevantes y el valor diferencial del proyecto.

2.1. PRECEDENTES

- Antecedentes teóricos

Se realizó una recopilación exhaustiva de documentación sobre los fundamentos de las ciencias de la computación y la teoría de lenguajes formales [1] [2]. Entre los temas estudiados se incluyeron:

- Historia y fundamentos: computabilidad y complejidad computacional.
- Lenguajes formales y gramáticas: símbolos, alfabetos, palabras, operaciones con lenguajes, gramáticas formales y jerarquía de chomsky: gramáticas tipo 0 (no restringidas), tipo 1 (sensibles al contexto), tipo 2 (independientes del contexto) y tipo 3 (regulares), normalización (FNC y FNG), ambigüedad y árboles de derivación.
- Autómatas finitos: AFD, AFND, equivalencia y minimización.
- Autómatas a pila: Relación con gramáticas independientes del contexto, aceptación por vaciado o estados finales y transformación entre APV y APF.
- Máquinas de Turing: Operaciones básicas, variaciones y problemas irresolubles como el problema de parada.

Este análisis permitió construir una base sólida de conocimiento para entrenar el modelo generativo utilizado en Fluxo, validar su capacidad para responder preguntas complejas y crear problemas solucionados correctamente que sirvieran de guía y apoyo al asistente virtual.

- Investigación de herramientas similares

Se realizó un análisis de herramientas existentes con características similares a las funcionalidades previstas para Fluxo, con el objetivo de identificar buenas prácticas y elementos clave que pudieran incorporarse al desarrollo del proyecto. Las herramientas investigadas fueron: *JFLAP*, *ToTalf*, *Automata Tutor* y *Khanmigo*.

- Bases de datos vectoriales y procesamiento de archivos

Se llevó a cabo un estudio detallado sobre el funcionamiento de sistemas basados en la técnica de RAG, que combina la recabación eficiente de datos relevantes con la generación de texto mediante LLMs. Este enfoque utiliza modelos de *embeddings* para transformar datos textuales en representaciones vectoriales densas, que capturan relaciones semánticas y contextuales, superando las limitaciones impuestas por el número máximo de *tokens* procesables por el modelo generativo. Esto permitió que el sistema accediera únicamente a los fragmentos relevantes en cada consulta, evitando cargar toda la base de conocimiento y favoreciendo tanto la escalabilidad del proyecto como la incorporación futura de nuevos documentos sin necesidad de realizar un *fine-tuning*, lo cual hubiera implicado un elevado coste económico.

El análisis incluyó desde la generación inicial de archivos en formato *Markdown* hasta su conversión a *JSON*, siguiendo un esquema estructurado de tipo *Problem-Solution*, seleccionado tras revisar el artículo técnico *Mat LLM*. Este formato fue identificado como ideal para mejorar la claridad, accesibilidad y organización de los datos, especialmente en aplicaciones educativas. [3]

Metamath Theorem Proving	MATH Dataset (Ours)
To prove: $n \in \mathbb{N} \wedge \frac{n+1}{2} \in \mathbb{N} \implies \exists m \in \mathbb{N} : n = 2m + 1$. GPT-<i>f</i>'s generated proof: <pre> - ((N e. NN0 /\ ((N + 1)/2) e. NN0) -> ((N - 1) / 2) e. NN0) - (N e. NN0 -> N e. CC) - 1 e. CC - ((N e. CC /\ 1 e. CC) -> (N - 1) e. CC) : </pre>	Problem: Tom has a red marble, a green marble, a blue marble, and three identical yellow marbles. How many different groups of two marbles can Tom choose? Solution: There are two cases here: either Tom chooses two yellow marbles (1 result), or he chooses two marbles of different colors ($\binom{4}{2} = 6$ results). The total number of distinct pairs of marbles Tom can choose is $1 + 6 = \boxed{7}$. Problem: If $\sum_{n=0}^{\infty} \cos^{2n} \theta = 5$, what is $\cos 2\theta$? Solution: This geometric series is $1 + \cos^2 \theta + \cos^4 \theta + \dots = \frac{1}{1 - \cos^2 \theta} = 5$. Hence, $\cos^2 \theta = \frac{4}{5}$. Then $\cos 2\theta = 2 \cos^2 \theta - 1 = \boxed{\frac{3}{5}}$. Problem: The equation $x^2 + 2x = i$ has two complex solutions. Determine the product of their real parts. Solution: Complete the square by adding 1 to each side. Then $(x + 1)^2 = 1 + i = e^{\frac{i\pi}{4}} \sqrt{2}$, so $x + 1 = \pm e^{\frac{i\pi}{8}} \sqrt[4]{2}$. The desired product is then $(-1 + \cos(\frac{\pi}{8}) \sqrt[4]{2})(-1 - \cos(\frac{\pi}{8}) \sqrt[4]{2}) = 1 - \cos^2(\frac{\pi}{8}) \sqrt{2} = 1 - \frac{(1 + \cos(\frac{\pi}{4}))}{2} \sqrt{2} = \boxed{\frac{1 - \sqrt{2}}{2}}$.
DeepMind Mathematics Dataset	
Problem: Divide 1136975704 by -142121963 Answer: -8 Problem: Calculate $((-2)/3)/(-1-(-24)/9)$ Answer: -2/5 Problem: Let $k(u) = u**2+u-4$. Find $k(0)$ Answer: -4 Problem: Sort 2, 4, 0, 6 Answer: 0, 2, 4, 6 Problem: Solve $4 - 4 - 4 = 188*m$ for m Answer: -1/47	

Figura 1: Comparación Datasets

En cuanto al almacenamiento, se evaluaron diversas opciones para bases de datos vectoriales y se seleccionó *Qdrant* por ser una solución gratuita, *open source* y perfectamente adaptada a las necesidades del proyecto. Su capacidad para realizar búsquedas eficientes, gestionar datos vectoriales en alta dimensionalidad y ofrecer escalabilidad garantizó un sistema flexible y robusto para alojar los datos procesados mediante RAG.

2.2. TECNOLOGÍAS APLICADAS

- *Frameworks* para desarrollo de aplicaciones web interactivas [4]

Se investigaron diversas alternativas para el desarrollo de interfaces de usuario, centrándose en *frameworks* y herramientas orientadas a aplicaciones de ciencia de datos e inteligencia artificial. El análisis incluyó soluciones como *Streamlit*, *Flask* y *Java*, evaluando su idoneidad para la integración de componentes de procesamiento de lenguaje natural, la gestión eficiente de estados de aplicación, la creación de interfaces de respuesta y la conexión con *APIs* externas.

Se valoraron especialmente aquellos que ofrecían componentes reutilizables, soporte multiplataforma y facilidad de integración con servicios de IA y bases de datos vectoriales. Sin embargo, la decisión final de utilizar *Streamlit* estuvo motivada principalmente por la familiaridad previa con la herramienta y por recomendación directa, además de ser una de las opciones permitidas para el despliegue en *Hugging Face Spaces*, lo que facilitó la publicación y acceso multiplataforma de la aplicación sin requerir conocimientos avanzados de *frontend* ni instalaciones adicionales.

- Plataformas de inferencia para modelos de lenguaje [5]

Se realizó un estudio comparativo de diferentes servicios para la ejecución de *LLMs*, considerando aspectos como latencia, rendimiento, costes y limitaciones de uso. La investigación abarcó tanto soluciones basadas en *GPU* tradicionales como arquitecturas especializadas, destacando plataformas como *GroqCloud*, *SambaNova*, y *AWS*. Se priorizaron aquellas capaces de ejecutar eficientemente modelos de la familia Llama con baja latencia, permitiendo mantener conversaciones fluidas y generar respuestas contextualizadas en tiempo real. *GroqCloud* fue seleccionada por su tecnología de inferencia basada en *LPU*, que proporciona velocidades de respuesta significativamente superiores a las de otras plataformas, con soporte para modelos *Llama 4* y precios competitivos, además de ofrecer un nivel gratuito adecuado para proyectos educativos.

- Selección de un modelo óptimo de IA generativa [6]

Dado que la plataforma de inferencia elegida fue *GroqCloud*, la selección del modelo de IA generativa se restringió a los modelos gratuitos disponibles en dicha plataforma. Se consultaron fuentes especializadas para comparar el rendimiento, capacidades y limitaciones de los modelos ofrecidos. El modelo *Llama 4 Maverick* fue seleccionado por su equilibrio entre calidad de generación, razonamiento, manejo de contexto extenso y velocidad de inferencia en *GroqCloud*, según análisis independientes y rankings de modelos publicados en plataformas como *Artificial Analysis* y *Hugging Face Models*. Esta elección garantizó la disponibilidad gratuita y la posibilidad de integración sencilla mediante *API*.

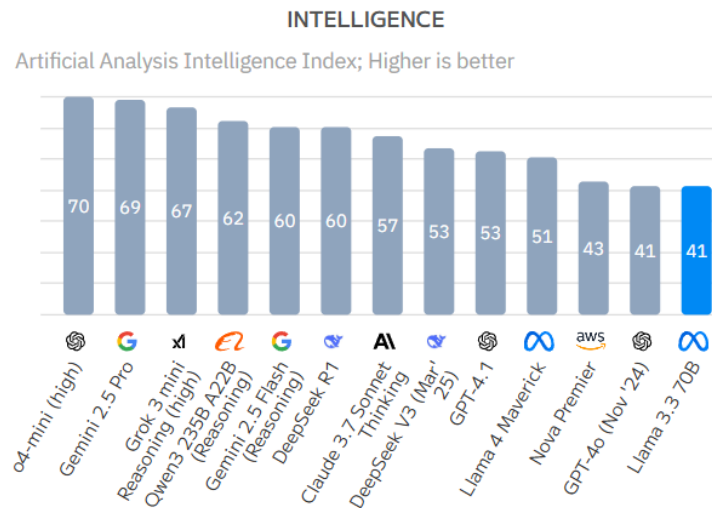


Figura 2: Inteligencia

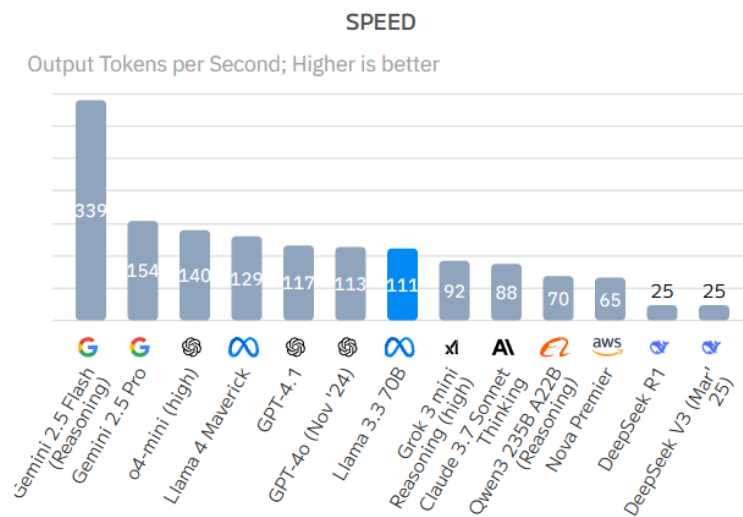


Figura 3: Velocidad

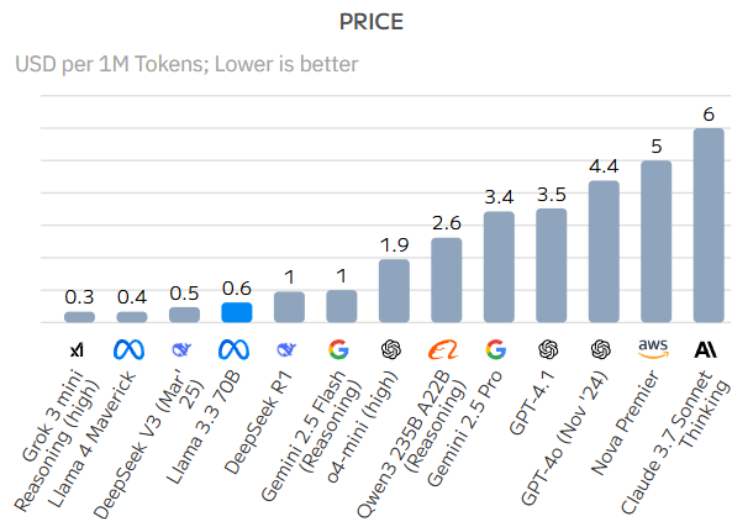


Figura 4: Precio

- Bases de datos vectoriales [7]

Se realizó un análisis comparativo de diferentes soluciones para el almacenamiento y recuperación eficiente de *embeddings*, elemento fundamental en sistemas basados en RAG. El estudio contempló factores como el rendimiento en búsquedas de similitud mediante distancia coseno, capacidad de escalabilidad, mecanismos de indexación, integración con distintos tipos de *embeddings* y facilidad de uso mediante *APIs*. Asimismo, se evaluó el soporte para metadatos asociados a vectores, opciones de filtrado por atributos y la viabilidad de implementación en entornos con recursos limitados. Finalmente, se optó por *Qdrant* como base de datos vectorial, dada su eficiencia, escalabilidad, compatibilidad con los flujos de trabajo requeridos y gran capacidad gratuita.

- Entornos de despliegue para aplicaciones de IA

Se examinaron diversas plataformas para el alojamiento y distribución de aplicaciones basadas en inteligencia artificial, priorizando soluciones que facilitaran el acceso multiplataforma sin requerir instalación local. El análisis incluyó servicios como *Hugging Face Spaces*, *OpenAI* y *Anthropic*, valorando la facilidad de configuración, integración con servicios externos, disponibilidad de recursos computacionales para múltiples usuarios simultáneos y opciones gratuitas para proyectos educativos. Se eligió *Hugging Face Spaces* por su integración nativa con aplicaciones *Streamlit*, soporte para bases de datos vectoriales y modelos de lenguaje, así como su accesibilidad multiplataforma y ausencia de requisitos de instalación, lo que garantizó una experiencia de usuario óptima y un despliegue sencillo de uso.

2.3. VALOR DIFERENCIAL DEL PROYECTO

Fluxo destacó como una aplicación web innovadora en el ámbito educativo al integrar dos módulos principales: un *chatbot* especializado para la consulta de documentación técnica y un *copilot* educativo que ofreció retroalimentación detallada y personalizada sobre la resolución de problemas. A diferencia de otras herramientas, Fluxo fue capaz de adaptarse a problemas nuevos, identificar errores concretos y guiar al usuario hacia una comprensión más profunda, superando los enfoques tradicionales basados en respuestas predefinidas. Esta combinación permitió ofrecer una experiencia de aprendizaje activa, adaptativa y diferenciada para el estudio de gramáticas y autómatas.

3. OBJETIVOS

3.1. OBJETIVO GENERAL

Desarrollo de una aplicación web accesible, basada en un sistema *RAG*, que permitiera ofrecer asistencia virtual especializada en gramáticas regulares, gramáticas independientes de contexto y autómatas finitos, facilitando el aprendizaje y la resolución de problemas complejos tanto para estudiantes como para docentes.

Por ello, el asistente virtual se concibió para complementar y superar estas carencias, integrando funcionalidades que permitieran:

1. Analizar soluciones paso a paso: El asistente virtual debía evaluar cada etapa del proceso realizado por el usuario al resolver un problema.
2. Identificar errores específicos: En lugar de limitarse a indicar si una solución es correcta o incorrecta, debía señalar los errores concretos cometidos en cada paso.
3. Sugerir mejoras: Proporcionar recomendaciones claras y precisas para corregir los errores detectados, o mejorar la solución existente.
4. Guiar hacia una comprensión más profunda: Ayudar al usuario a entender los conceptos subyacentes al problema planteado, fomentando un aprendizaje activo y constructivo.

3.2. OBJETIVOS ESPECÍFICOS

1. Desarrollar una base de datos de conocimiento completa: Crear un *dataset* de archivos *Markdown* que abarcara todos los temas imprescindibles sobre lenguajes formales, gramáticas y autómatas, incluyendo teoría fundamental y problemas resueltos. Posteriormente, convertir esta documentación a una base de datos vectorial en *Qdrant* mediante el proceso de generación de *embeddings* para su recuperación semántica.
2. Desarrollar una aplicación web multiplataforma: Construir *Fluxo* utilizando *Streamlit* como *framework* principal, integrando las funcionalidades del *chatbot* especializado y el *copilot* educativo en una interfaz accesible desde navegadores, con conexión al modelo *Llama 4 Maverick* mediante *GroqCloud*.
3. Implementar un sistema *RAG* eficiente: Desarrollar un sistema que unificara todos los componentes de *Fluxo*, integrando la interfaz de usuario (*Hugging Face Spaces*), el

procesamiento de consultas, la recuperación de información relevante (*Qdrant*) y la generación de respuestas (*Groq*), asegurando un flujo adecuado entre todos estos elementos.

4. Garantizar accesibilidad y usabilidad: Desplegar la aplicación en *Hugging Face Spaces* para permitir acceso universal sin instalación, acompañado de una guía de uso que facilite la adopción por parte de estudiantes y docentes en entornos educativos diversos.
5. Validar la efectividad del sistema: Realizar pruebas exhaustivas para evaluar la precisión semántica del *chatbot*, la efectividad pedagógica del *copilot*, el rendimiento del sistema *RAG* y la escalabilidad técnica al incorporar nuevos documentos a la base de conocimiento.

3.3. MÉTODOS DE VALIDACIÓN

Validación de la base de conocimiento

Se validó uno a uno la veracidad y corrección de los ejercicios de referencia, reflejando todo el proceso en un archivo para poder hacer seguimiento de los errores, de esta forma, este archivo llamado *Revision*, se usaría para validar el funcionamiento de *Fluxo*, comparándolo con IAs ya existentes.

Validación del RAG

El correcto funcionamiento del *RAG* se verificó mediante los siguientes criterios:

- Límite de tokens: Se comprobó que consulta + documentos relevantes no superaba el límite estipulado de 6000 *tokens* por minuto (*input + output*), dado que es la capacidad gratuita. [8]
- Extracción precisa de información: Se validó que el *RAG* se alimentara correctamente de los archivos desarrollados en formato *Markdown*. Para ello, *Fluxo* debía adjuntar en sus respuestas un enlace directo al archivo específico del cual extrajera la información, permitiendo rastrear y verificar la fuente utilizada, asegurando así que el modelo capturara adecuadamente el significado semántico de los documentos, permitiendo búsquedas eficientes.
- Escalabilidad: Se añadieron archivos posteriormente, verificando la capacidad del sistema para manejar un mayor volumen de datos o consultas sin pérdida significativa de rendimiento.

Validación del funcionamiento general de Fluxo

El correcto desempeño de *Fluxo* como asistente virtual se evaluó mediante pruebas exhaustivas realizadas en colaboración con el tutor del TFG. Estas pruebas incluyeron:

- Validación de respuestas: Se analizaron las respuestas generadas por *Fluxo* (tanto del *chatbot* como del *copilot*) para garantizar que fueran correctas, relevantes y útiles

desde un punto de vista pedagógico, comparándolas con los errores detectados en el archivo Revisión y con los errores detectados por una IA de pago.

- Retroalimentación granular: En el caso del *copilot* educativo, también se verificó que las sugerencias ofrecidas fueran claras, detalladas y orientadas a guiar al usuario hacia una comprensión más profunda del problema planteado.

Validación del despliegue multiplataforma

Para evaluar la accesibilidad y usabilidad de la solución web:

- Funcionamiento en diferentes dispositivos: Se comprobó que la aplicación funcionara correctamente en diversos navegadores, sistemas operativos y tamaños de pantalla, garantizando su carácter multiplataforma.
- Prueba con usuarios externos: Se invitó a usuarios externos a acceder a Fluxo a través de su enlace en *Hugging Face Spaces*, evaluando tanto la experiencia de usuario como la estabilidad de la aplicación bajo diferentes patrones de uso.
- Tiempo de respuesta: Se midieron los tiempos de respuesta del sistema completo, verificando que fueran adecuados para una interacción fluida, incluso con consultas complejas que requirieran procesamiento extenso.

La validación definitiva de Fluxo como herramienta educativa se completará cuando estudiantes y docentes comiencen a utilizarla regularmente en sus actividades académicas, es decir, cuando varias personas la usen a la vez, recogiendo su retroalimentación para futuras mejoras

4. PLAN DE DESARROLLO DEL PROYECTO

4.1. METODOLOGÍA

La metodología adoptada para el desarrollo de Fluxo se inspiró en el análisis de tres herramientas clave que cumplían funcionalidades similares a las previstas para el proyecto. Estas herramientas proporcionaron una base conceptual y técnica que guió las decisiones tomadas durante el diseño y la implementación:

- *Totalf* [9]: Esta herramienta fue fundamental para definir la estructura de la interfaz gráfica. Su desarrollo en *Java* motivó la elección de este lenguaje, ya que garantizaba una integración eficiente con *JFLAP* y su biblioteca programática (*jflap-lib*). Independientemente de que al final no se desarrollara en *Java*, se tomaron referencias sobre la organización visual y estructural de sus menús y botones, asegurando una experiencia de usuario intuitiva.

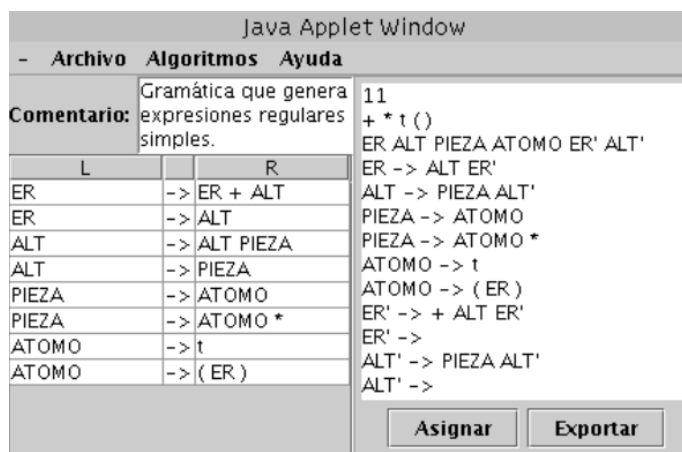


Figura 5: Funcionalidad de Totalf

- *Automata Tutor* [10]: Se utilizó como referencia para diseñar el sistema de retroalimentación. La forma en que *Automata Tutor* proporciona comentarios detallados y estructurados sobre los errores cometidos por los usuarios inspiró el desarrollo del *feedback* granular del asistente virtual, orientado a guiar paso a paso en la resolución de problemas.

- *Khanmigo* [11]: Esta herramienta basada en inteligencia artificial sirvió como modelo para conceptualizar las capacidades del *copilot* educativo. Su enfoque centrado en explicaciones detalladas y ayuda personalizada fue clave para diseñar un sistema adaptativo capaz de responder preguntas complejas y guiar al usuario en ejercicios prácticos.

Adicionalmente, el formato *Markdown* fue elegido para la creación inicial de la base de conocimiento siguiendo un consejo profesional recibido durante una estancia en Mapfre. Este formato destacó por su simplicidad, versatilidad y capacidad para estructurar información de manera clara, facilitando su posterior vectorización en formato *JSON* y su integración en el sistema *RAG*.

4.2. TECNOLOGÍAS

- *VS Code*: Se utilizó para la creación y edición de los archivos *Markdown* que conformaron la base de conocimiento inicial. La elección de este editor se debió a la familiaridad con su entorno y a su capacidad para generar documentación estructurada, facilitando la organización y posterior procesamiento de la información.
- *Streamlit*: Se empleó como *framework* principal para el desarrollo de la interfaz web de Fluxo, permitiendo la construcción de una aplicación interactiva y accesible desde cualquier navegador, sin requerir instalación local.
- *GroqCloud*: Plataforma seleccionada para la inferencia del modelo generativo *Llama 4 Maverick*, proporcionando respuestas rápidas y precisas a las consultas de los usuarios a través de su *API*. Su elección se basó en su rendimiento y en la facilidad de integración con aplicaciones web.
- *Qdrant*: Base de datos vectorial utilizada para almacenar y gestionar los *embeddings* vectoriales generados a partir de la base de conocimiento. Fue seleccionada por ser gratuita, de código abierto y especialmente adecuada para sistemas basados en *RAG*, permitiendo búsquedas semánticas eficientes y escalabilidad.
- *Hugging Face Spaces*: Plataforma de despliegue en la que se alojó la versión final de Fluxo, garantizando acceso multiplataforma y eliminando la necesidad de instalaciones locales.

4.3. PLAN DE DESARROLLO DEL PROYECTO

Plan de trabajo general

El plan de desarrollo se estructuró en varias fases principales, cada una con objetivos y entregables definidos, y se organizaron en paquetes de trabajo (PT) que guiaron la evolución del proyecto:

PT1: Investigación y recopilación de documentación

Objetivo: Construir una base sólida de conocimientos teóricos y técnicos para fundamentar el desarrollo.

Actividades:

Estudio detallado de lenguajes formales, gramáticas y autómatas.

Análisis de herramientas existentes para identificar funcionalidades clave.

Investigación sobre sistemas *RAG* y bases de datos vectoriales, incluyendo técnicas de *embeddings*.

Evaluación comparativa de tecnologías y *frameworks* aplicables.

PT2: Diseño conceptual del sistema

Objetivo: Definir la arquitectura general y las funcionalidades principales de Fluxo.

Actividades

Diseño del sistema *RAG* para gestionar la base de conocimiento mediante *Qdrant*.

Definición del formato estructurado *Problem-Solution* en *Markdown/JSON*.

Conceptualización del *copilot* educativo y el *chatbot* como módulos clave.

Diseño preliminar de la interfaz de usuario.

PT3: Desarrollo técnico

Objetivo: Implementar las funcionalidades principales utilizando las tecnologías seleccionadas.

Actividades:

Creación de la base de conocimiento en formato *Markdown*, incluyendo teoría y ejercicios resueltos.

Desarrollo de los algoritmos necesarios para la resolución de ejercicios, de tal forma que Fluxo se apoyara en ellos para verificar cada paso de la resolución.

Conversión de los archivos *Markdown* a *embeddings* vectoriales y almacenamiento en Qdrant.

Implementación del sistema *RAG* integrando la recuperación semántica y la generación de respuestas con *Llama 4 Maverick* a través de *GroqCloud*.

Desarrollo de la aplicación web con *Streamlit* y despliegue en *Hugging Face Spaces*.

PT4: Pruebas y validación

Objetivo: Garantizar que Fluxo cumpliera con los objetivos planteados mediante pruebas exhaustivas y evaluación continua.

Actividades:

Validación de la veracidad de la base de conocimiento.

Pruebas funcionales para verificar la precisión del *chatbot* y el *copilot* educativo.

Evaluación del rendimiento del sistema *RAG* en términos de tiempos de respuesta y relevancia de la información recuperada.

Validación supervisada para analizar la calidad de la retroalimentación y la utilidad de Fluxo como herramienta educativa.

PT5: Documentación final

Objetivo: Consolidar toda la información generada durante el proyecto en un documento final que reflejara el desarrollo realizado.

Actividades:

Redacción detallada de la memoria final, incluyendo antecedentes, metodología, tecnologías utilizadas, resultados y conclusiones.

4.3.1. Análisis y definición del problema

Se abordó la identificación y formalización del problema a resolver, estableciendo las bases teóricas y metodológicas necesarias para el desarrollo del proyecto:

- Revisión bibliográfica y análisis del estado del arte:

Se realizó un estudio exhaustivo de herramientas existentes relacionadas con la enseñanza de lenguajes formales y autómatas, como *Totalf*, *Automata Tutor* y *Khanmigo*. Este análisis permitió identificar las limitaciones de estas herramientas, como la falta de retroalimentación personalizada o la dependencia de casos predefinidos, lo que justificó la necesidad de una solución más flexible e innovadora basada en modelos de lenguaje avanzados. La revisión incluyó también el análisis de *JFLAP* como herramienta de referencia, identificando sus fortalezas para la representación gráfica y sus carencias en cuanto a orientación pedagógica interactiva.

- Definición de principios técnicos y metodológicos:

Se definieron los principios fundamentales para el diseño del sistema RAG, integrando una aplicación web (*Streamlit*) desplegada en *Hugging Face Spaces*, una base de datos vectorial (*Qdrant*) y un modelo generativo (*Llama 4 Maverick*) mediante *GroqCloud*. Este enfoque permitió superar las limitaciones de los modelos generativos tradicionales al incorporar recuperación semántica de información relevante, mejorando la precisión y contextualización de las respuestas.

- Especificación de objetivos y criterios de evaluación:

Se establecieron los objetivos específicos del proyecto, incluyendo el desarrollo de una base de conocimiento completa, la implementación de la aplicación web multiplataforma y la integración de un sistema RAG eficiente que unificara todos los componentes mostrados en el diagrama. Los criterios para evaluar el éxito del proyecto incluyeron:

- Precisión en las respuestas generadas por el *chatbot*.
- Calidad del *feedback* proporcionado por el *copilot* educativo.
- Eficiencia en la recuperación de información mediante el sistema RAG.
- Escalabilidad y facilidad para integrar nuevos datos en el sistema.
- Accesibilidad y usabilidad de la solución implementada.

4.3.2. Desarrollo del modelo matemático

El desarrollo del modelo matemático del sistema RAG que sustenta Fluxo se estructuró en tres componentes fundamentales:

Formulación de ecuaciones y técnicas matemáticas

1. Generación de *embeddings* mediante *Mean Pooling*:
La transformación de documentos textuales a representaciones vectoriales densas se implementó mediante la ecuación:

$$E(d) = \frac{\sum_{i=1}^n (h_i \cdot m_i)}{\sum_{i=1}^n m_i}$$

Componente	Rol	Ejemplo
E(d)	<i>Embedding</i> del fragmento (384 dimensiones, $-1 < x < 1$)	[0.12, -0.45, ..., 0.98]
hi	Vector de estado oculto del <i>token i</i> (salida de la capa <i>embedding</i> del <i>transformer</i>)	Tensor de 384 valores
mi	Máscara binaria (1 = token válido, 0 = padding)	[1, 1, ..., 0, 0]
n	Número máximo de <i>tokens</i> por fragmento (ajuste dinámico ≤ 512)	500 <i>tokens</i> base (400 reales + 100 <i>pooling</i>) + 12 especiales

Tabla 1: Componentes Mean Pooling

El modelo procesó cada documento respetando el límite máximo de 512 *tokens* por fragmento impuesto por la arquitectura de *BERT*, que utilizó *embeddings* posicionales preentrenados únicamente para las primeras 512 posiciones. Para generar representaciones vectoriales, se aplicó la técnica de *mean pooling*, que sumó los vectores ocultos de cada token (*hi*) ponderándolos por su máscara de atención binaria (*mi*), que identifica *tokens* válidos (*mi*=1) vs *padding* o relleno (*mi*=0). Posteriormente se dividió por el total de *tokens* válidos.

hi es un vector denso de 384 dimensiones que representa la información semántica y contextual de ese *token*, que se obtuvo en la capa de *embedding* del *transformer*.

Este proceso produjo un único vector denso de 384 dimensiones por fragmento, cada una de las 384 columnas de esa fila contenía un valor numérico que representaba una característica semántica particular del fragmento en el espacio vectorial definido por el modelo *all-MiniLM-L6-v2*, se optimizó para mantener el equilibrio entre rendimiento y eficiencia. Los documentos largos se dividieron automáticamente en fragmentos consecutivos, cada uno procesado de forma independiente, garantizando que ninguna secuencia superara el límite contextual del modelo.

2. Búsqueda por similitud coseno:

$$\text{similarity}(q, di) = \frac{q \cdot di}{||q|| \cdot ||di||}$$

La función de similitud coseno compara matemáticamente la orientación semántica entre dos vectores: el de la consulta del usuario (*q*) y el de cada fragmento (*di*). Este cálculo, que equivale al coseno del ángulo formado por ambos vectores en el espacio multidimensional, generó valores entre -1 (vectores semánticamente opuestos) y 1 (vectores idénticos en significado).

Para garantizar precisión en la comparación, se aplicó previamente un proceso de normalización vectorial, cada vector se ajustó a longitud unitaria y posteriormente se realizó el producto escalar.

El algoritmo *KNN* utilizó estos valores de similitud para seleccionar los 3 fragmentos ($k=3$) semánticamente más próximos a la consulta, independientemente del documento original al que pertenezcan, priorizando la relevancia contextual. Se utilizó *KNN* pero en su variante optimizada mediante *HNSW* para garantizar eficiencia en grandes volúmenes de datos.

.

3. Arquitectura de flujo secuencial:
El modelo completo siguió un proceso de 7 pasos interconectados, expresados matemáticamente como transformaciones secuenciales desde la consulta inicial hasta la respuesta final.

Implementación mediante software especializado

La implementación del modelo se realizó en *Python*, utilizando bibliotecas especializadas:

- Procesamiento y vectorización de documentos:
Se utilizó la biblioteca *SentenceTransformer* con el modelo preentrenado '*all-MiniLM-L6-v2*' para transformar documentos *Markdown* en *embeddings* vectoriales de 384 dimensiones. Este proceso se aplicó a la base de conocimiento, dividiendo los documentos en fragmentos semánticamente coherentes basados en la estructura de encabezados.
- Sistema de almacenamiento vectorial:
Se configuró *Qdrant* para crear colecciones especializadas tanto para documentación técnica como para ejercicios resueltos, utilizando la distancia coseno como métrica de similitud. Este componente permitió indexar y almacenar los vectores generados, facilitando consultas de similitud semántica eficientes, recuperando los fragmentos más relevantes de la base de conocimiento, limitando los resultados a los tres documentos más relevantes.
- Integración con el modelo generativo:
Se desarrolló la conexión con el modelo *Llama 4 Maverick* a través de *GroqCloud*, enviando consultas estructuradas que combinaban la pregunta original con los documentos recuperados, siguiendo un formato optimizado para obtener respuestas contextualizadas.
- Despliegue en *Hugging Face Spaces*: Se creó un espacio público empleando *Streamlit* como *framework* principal, lo que permitió alojar la versión definitiva de la aplicación y garantizar su accesibilidad desde cualquier dispositivo y sistema operativo, sin necesidad de instalación local.

Validación preliminar del modelo

La validación del modelo matemático se realizó mediante pruebas sistemáticas:

- Precisión de recuperación: Se verificó que el sistema recuperara documentos semánticamente relevantes para diversas consultas de prueba, evaluando tanto la calidad como la relevancia de los resultados.
- Consistencia de *embeddings*: Se comprobó que fragmentos de texto con similitud semántica generaran *embeddings* con alta similitud coseno, confirmando la efectividad de la representación vectorial.

- Rendimiento del flujo completo: Se evaluó el tiempo de procesamiento en cada paso del diagrama, desde la consulta inicial hasta la entrega de la respuesta, verificando la viabilidad del sistema en un entorno de producción.

4.3.3. Implementación y análisis de resultados

Este paquete estuvo enfocado en la implementación completa del sistema y en la evaluación de su desempeño mediante pruebas exhaustivas.

- Ajuste y optimización del sistema:

Se realizaron iteraciones sobre el diseño inicial del sistema *RAG* para mejorar su eficiencia y precisión. Esto incluyó ajustes en los parámetros utilizados para generar *embeddings*, como el tamaño de los *chunks*, así como optimizaciones en la integración entre *Qdrant* y *Groq*. También se optimizó la interfaz de usuario desarrollada en *Streamlit* para garantizar una experiencia fluida y accesible desde cualquier dispositivo.

- Aplicación a escenarios prácticos:

Se probó Fluxo con problemas reales relacionados con lenguajes formales y autómatas, evaluando su capacidad para proporcionar retroalimentación personalizada ante problemas nuevos no contemplados previamente en la base de conocimiento.

- Análisis e interpretación de resultados:

Los resultados obtenidos fueron analizados tanto desde una perspectiva técnica como educativa:

- Desde el punto de vista técnico, se evaluaron métricas como tiempos de respuesta (desde la consulta inicial hasta la presentación del resultado final), relevancia semántica de la información recuperada y estabilidad de la aplicación desplegada.
- Desde el punto de vista educativo, se analizó la utilidad práctica del *feedback* proporcionado por el *copilot*, así como su capacidad para guiar al usuario en ejercicios complejos.

4.4. PLAN DE TRABAJO

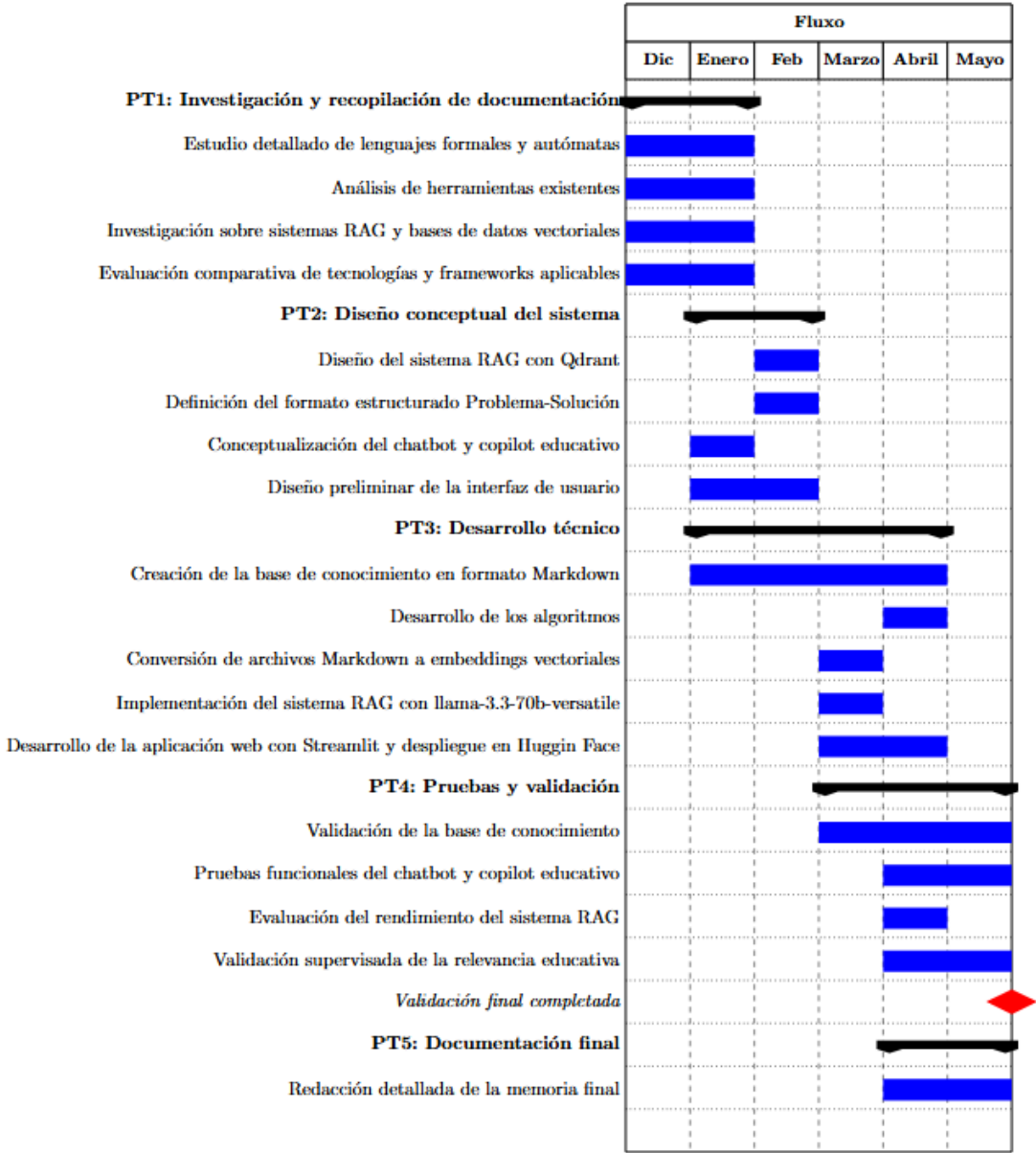


Figura 6: Diagrama de Gantt.

4.5. RECURSOS.

Recursos técnicos

- Plataforma de inferencia *GroqCloud*: Se empleó para acceder al modelo de inteligencia artificial generativa *Llama 4 Maverick*, proporcionando respuestas rápidas y precisas a las consultas de los usuarios a través de su *API*.
- *Qdrant*: Base de datos vectorial utilizada para implementar el sistema *RAG*, permitiendo el almacenamiento y búsqueda eficiente de *embeddings* mediante la métrica de similitud coseno. [12]
- *Streamlit*: *Framework* principal para el desarrollo de la aplicación web, proporcionando componentes interactivos para la interfaz de usuario y facilitando la integración con servicios externos como *GroqCloud* y *Qdrant*.
- *Hugging Face Spaces*: Plataforma de despliegue utilizada para alojar la aplicación web y hacerla accesible desde cualquier dispositivo sin necesidad de instalación local.
- *Sentence-Transformers*: Biblioteca especializada empleada para generar *embeddings* vectoriales a partir de la documentación textual, utilizando específicamente el modelo *all-MiniLM-L6-v2*.
- *Python*: Lenguaje de programación utilizado para desarrollar todos los componentes del sistema, incluyendo el procesamiento de documentos, la generación de *embeddings*, el modelo y la aplicación web.
- *Markdown/JSON*: Formatos utilizados para estructurar la base de conocimiento, incluyendo teoría y ejercicios prácticos.

Colaboradores

El proyecto contó con la colaboración directa e indirecta de personas clave que aportaron conocimiento técnico, asesoramiento y supervisión:

- Tutor académico: Juan José Escribano desempeñó un papel crucial en la orientación del proyecto, ofreciendo apoyo en la recopilación de documentación, evaluación del copilot educativo y refinamiento de la base de conocimiento.
- Equipo técnico externo: Elena Gutiérrez Cruz, experta en IA de Mapfre, asesoró en la implementación del modelo y posteriormente sobre el funcionamiento y estructura de las bases de datos vectoriales y sistemas *RAG*.
- Profesores especializados: Moisés Martínez, experto en IA, aportó documentación relevante para el estado del arte, como la estructura *Problem-Solution*, así como la plataforma de despliegue *Hugging Face*.

4.6. COSTES

Actualmente, Fluxo utiliza el modelo *Llama 4 Maverick* en su versión gratuita, lo que permitía ofrecer un asistente virtual avanzado. Este modelo destaca por ser un modelo de lenguaje abierto y eficiente, capaz de entender y generar texto de calidad, disponible como software libre para la mayoría de los usuarios.

Para la funcionalidad de consulta de documentación se empleó específicamente el modelo *llama-3.3-70b-versatile*. Esta decisión permitió que ambas funcionalidades funcionaran de manera independiente, optimizando la capacidad global del sistema, ya que cada módulo operaba con sus propias limitaciones y recursos. De esta forma, la consulta de documentación no interfería con la resolución de ejercicios y viceversa, lo que resultó especialmente relevante en escenarios de uso concurrente o intensivo.

En cuanto a las limitaciones, *llama-3.3-70b* presentaba restricciones propias en cuanto a número de *tokens* procesables por minuto y concurrencia de peticiones, pero ofrecía una excelente relación entre calidad de respuesta y eficiencia para tareas de recuperación y explicación teórica. Por su parte, *Llama 4 Maverick* estaba optimizado para la generación de texto más compleja y el análisis detallado de soluciones propuestas por los usuarios, aunque presentaba las mismas limitaciones de *tokens*.

Sin embargo, si se pretendía dar un salto cualitativo en la calidad de las respuestas, la robustez ante consultas complejas y la capacidad de atender a múltiples usuarios simultáneamente, compensaba considerar la contratación de un modelo aún más potente o de acceso *premium*. El uso de modelos superiores implicaba una inversión adicional, pero aportaba ventajas como mayor velocidad de inferencia, mejor comprensión contextual y soporte técnico profesional.

Principales costes y consideraciones de escalabilidad asociados a la infraestructura actual y a una posible evolución del sistema:

- Tokens en GroqCloud:
El modelo generativo empleado, en su versión más potente, cobra por token procesado. Para *Llama 4 Maverick*, el precio ronda los \$0.20 por millón de tokens de entrada y \$0.60 por millón de tokens de salida, lo que equivale aproximadamente a 0,4 € por 1M tokens conjunto, dependiendo del volumen y la complejidad de las consultas. Por otro lado, *llama 3.3-70b versatile* el precio ronda los \$0.59 por millón de tokens de entrada y \$0.79 por millón de tokens de salida, lo que equivale aproximadamente a 0,64 € por 1M tokens conjunto. [3]

La estimación final del coste depende del número de alumnos o usuarios concurrentes, por lo que un uso intensivo o institucional requiere monitorización y previsión presupuestaria.

Esta arquitectura modular, basada en el uso diferenciado de *llama-3.3-70b* para la consulta de documentación y *Llama 4 Maverick* para el copilot educativo, permitió maximizar el rendimiento y la escalabilidad de Fluxo, adaptándose a las necesidades reales de los usuarios y garantizando la independencia funcional de cada módulo.

- Almacenamiento en Qdrant:

Qdrant ofrece un nivel gratuito con 1 GB de almacenamiento, suficiente para la fase inicial del proyecto. Si la cantidad de datos almacenados aumenta significativamente, sería necesario contratar espacio adicional. Los precios variaban según el proveedor y la capacidad requerida, aunque en la situación actual esto no suponía una limitación relevante. [12]

- Infraestructura en Hugging Face Spaces [14]:

Fluxo está desplegado en Hugging Face Spaces utilizando el nivel gratuito (*CPU Basic*: 2 vCPU, 16 GB RAM, 50 GB de almacenamiento no persistente), lo que permite el acceso multiplataforma y sin instalación para los usuarios. No obstante, para escalar a múltiples usuarios concurrentes o mejorar el rendimiento, se debería considerar la actualización a opciones de pago, tales como:

- CPU mejorada (desde \$0.03/hora)
- Opciones GPU (desde \$0.60/hora hasta \$10.80/hora)
- Almacenamiento persistente (desde \$5/mes por 20 GB adicionales hasta \$100/mes por 1 TB)

Condicionantes y Limitaciones

Limitaciones económicas

- *GroqCloud*: En la versión gratuita para el modelo *Llama 4 Maverick*, el límite actual es de 30 peticiones por minuto, 1000 por día y 6.000 *tokens* por minuto y sin límite de *tokens* diarios. Por otro lado, *llama-3.3-70b*, presenta un límite de 30 peticiones por minuto, 1000 por día y 12.000 *tokens* por minuto y 100.000 por día. Aunque estos valores han aumentado respecto a versiones anteriores, siguen representando una restricción para la escalabilidad en contextos con numerosos usuarios o consultas concurrentes. Para superar estas limitaciones, sería necesario migrar a un plan de pago (*Developer Tier*), que permite multiplicar por 10 los límites de uso y acceder a funcionalidades como el *Batch API* para procesamiento en paralelo, lo que implica un coste adicional por uso y almacenamiento de *tokens*. [8]
- *Fine-tuning* del modelo: Se descartó debido a los elevados recursos computacionales y económicos que requiere. Este proceso demanda hardware especializado (*GPUs* de alto rendimiento) y tiempos prolongados de entrenamiento, lo que incrementa notablemente los costes. Además, cada actualización de la base de conocimiento habría requerido un nuevo proceso de reentrenamiento.
- Almacenamiento y despliegue: *Qdrant* y *Hugging Face Spaces* ofrecen modalidades gratuitas que cubrieron las necesidades del desarrollo. Sin embargo, si el volumen de datos o el número de usuarios aumentara significativamente, sería necesario contratar planes de pago para almacenamiento adicional en *Qdrant* o para mejorar la infraestructura de *Hugging Face Spaces*.

Condicionantes técnicos

- Integración con *JFLAP*: La interfaz y arquitectura de *JFLAP* presentaron inicialmente limitaciones que impidieron la integración directa de las funcionalidades avanzadas del *chatbot* y el *copilot* educativo dentro de la propia herramienta. Por este motivo, en una primera fase, Fluxo se desarrolló como un entorno web independiente.

Posteriormente, se abordó el desarrollo de una solución en *Java* con *JFLAP* integrado, con el objetivo de convertir Fluxo en un componente .jar descargable que unificara ambas herramientas en una sola aplicación multiplataforma. Sin embargo, esta alternativa tampoco resultó viable debido a las dificultades técnicas y de distribución asociadas a la creación de un ejecutable descargable y a la compatibilidad multiplataforma. Finalmente, se optó por la solución web definitiva, basada en *Streamlit* y desplegada en *Hugging Face Spaces*, que permitió mantener todas las funcionalidades avanzadas del asistente virtual y garantizar el acceso multiplataforma sin necesidad de instalación.

5. DESARROLLO DE LA SOLUCIÓN TÉCNICA

5.1. PT1 INVESTIGACIÓN Y RECOPIACIÓN DE INFORMACIÓN

5.1.1. Estudio detallado de lenguajes formales, gramáticas y autómatas

Se llevó a cabo una revisión bibliográfica extensiva sobre los fundamentos de las ciencias de la computación y la teoría de lenguajes formales. Este proceso incluyó la recopilación, análisis y estructuración de información sobre:

- Fundamentos teóricos: Se estudiaron los conceptos básicos como símbolos, alfabetos, palabras y operaciones con lenguajes.
- Jerarquía de Chomsky: Se profundizó en los cuatro tipos de gramáticas (tipo 0 no restringidas, tipo 1 sensibles al contexto, tipo 2 independientes del contexto y tipo 3 regulares).

- **Autómatas finitos:** Se revisaron conceptos como determinismo, no determinismo, equivalencia y minimización.
- **Autómatas a pila:** Se analizó su relación con gramáticas independientes del contexto, aceptación por vaciado o estados finales.
- **Máquinas de Turing:** Se estudiaron sus operaciones básicas, variaciones y problemas irresolubles.

Este estudio permitió construir una base de conocimiento sólida que posteriormente se transformaría en la fuente principal para el sistema *RAG*. La documentación recopilada fue cuidadosamente organizada en archivos *Markdown*, estructurados de manera jerárquica para facilitar su posterior procesamiento y vectorización.

5.1.2. Análisis de herramientas existentes

Se realizó un estudio comparativo de herramientas relacionadas con la enseñanza de lenguajes formales y autómatas:

- ***JFLAP*:** Se analizó detalladamente esta herramienta ampliamente utilizada en entornos académicos, identificando sus fortalezas (visualización gráfica de autómatas y gramáticas) y sus limitaciones (falta de retroalimentación interactiva y orientación pedagógica limitada).
- ***Totalf*:** Se estudiaron las características de su interfaz y estructura, extrayendo referencias importantes para el diseño visual de *Fluxo*.
- ***Automata Tutor*:** Se analizó su sistema de retroalimentación, que sirvió como apoyo para el desarrollo del sistema de *feedback* granular del *copilot* educativo.
- ***Khanmigo*:** Se evaluaron sus capacidades como asistente educativo basado en IA, identificando características relevantes para el desarrollo del *chatbot* especializado.

Este análisis permitió identificar un nicho de oportunidad para *Fluxo*: la creación de un asistente virtual que combinara la retroalimentación personalizada con la capacidad de abordar problemas nuevos no predefinidos.

5.1.3. Investigación sobre sistemas *RAG* y bases de datos vectoriales

Se realizó un estudio exhaustivo sobre la técnica de *RAG*, que combina la recuperación eficiente de datos relevantes con la generación de texto mediante *LLMs*. Este proceso incluyó:

- **Estudio de *embeddings*:** Se investigó cómo transformar información textual en representaciones vectoriales densas, analizando diferentes modelos y técnicas.
- **Análisis de bases de datos vectoriales:** Se evaluaron múltiples soluciones (*Qdrant*, *Chroma*, *Pinecone*) para almacenar y recuperar *embeddings* de manera eficiente.
- **Procesamiento de documentos:** Se desarrollaron esquemas para convertir la documentación de formato *Markdown* a representaciones vectoriales, manteniendo la integridad semántica de los contenidos.

- Estudio de arquitecturas RAG existentes:

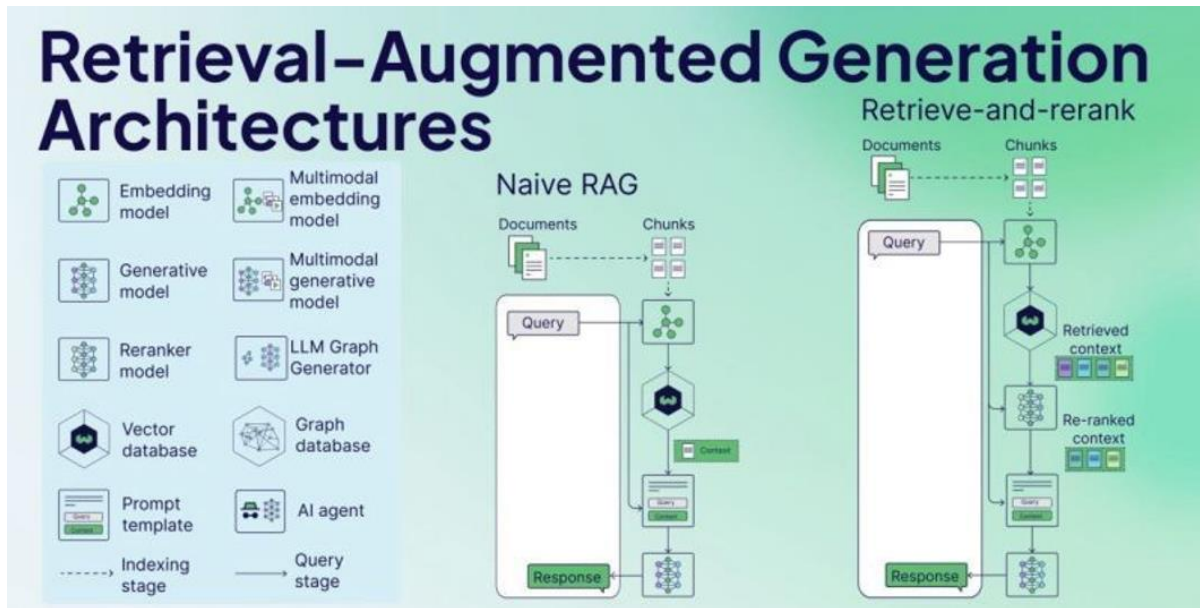


Figura 7: Arquitecturas RAG aplicables. [16]

La decisión de emplear la arquitectura *retrieve-and-rerank* en el desarrollo del asistente virtual se fundamentó en el análisis de las limitaciones de los modelos generativos tradicionales. La documentación consultada sobre esta técnica destaca varias ventajas clave que motivaron su adopción:

- Superación de la limitación de *tokens*: Los modelos generativos modernos, como los *LLMs*, presentan restricciones en la cantidad de *tokens* que pueden procesar en una sola consulta. La arquitectura *retrieve-and-rerank* permite seleccionar únicamente los fragmentos más relevantes de la base de conocimiento para cada pregunta, evitando la necesidad de cargar todo el corpus y garantizando que la respuesta generada esté respaldada por información precisa y contextualizada.
- Escalabilidad y actualización dinámica: Una de las principales fortalezas de este enfoque es la posibilidad de incorporar nueva documentación o actualizar la existente sin requerir el reentrenamiento del modelo generativo. Esto representa una ventaja significativa frente al *fine-tuning* tradicional, que implica altos costes computacionales y de mantenimiento cada vez que se modifica la base de conocimiento.
- Mejora de la precisión y relevancia: La combinación de recuperación semántica (*retrieval*) y *reranking* neural (*rerank*) incrementa la precisión de los sistemas de pregunta-respuesta, ya que el *reranker* evalúa el contexto de los fragmentos recuperados en función de la consulta, priorizando aquellos que mejor se ajustan a la intención del usuario.
- Flexibilidad y eficiencia: El sistema puede adaptarse a diferentes dominios y volúmenes de datos, manteniendo tiempos de respuesta aceptables y permitiendo la integración de nuevas fuentes de información de manera incremental y controlada.

La investigación realizada en el marco del proyecto incluyó el estudio de artículos y recursos especializados sobre sistemas *RAG* y bases de datos vectoriales, que evidenciaron que la

arquitectura *retrieve-and-rerank* es un estándar en aplicaciones educativas y empresariales que requieren respuestas precisas y actualizables a partir de grandes volúmenes de documentación. [13] [17] [18]

5.1.4. Evaluación comparativa de tecnologías y *frameworks*

Se realizó un análisis detallado de las diferentes tecnologías y *frameworks* que podrían utilizarse para implementar Fluxo:

- **Frameworks para desarrollo web:** Se evaluaron alternativas como *Streamlit*, *java* y *Flask*, seleccionando finalmente *Streamlit* por su facilidad de uso, integración con componentes de IA y capacidad para desarrollar interfaces interactivas con poco código, así como por recomendación y familiaridad.
- **Plataformas de inferencia LLM:** Se compararon diferentes proveedores de servicios de inferencia para modelos de lenguaje, incluyendo *GroqCloud*, *SambaNova*, y *AWS*, seleccionando *GroqCloud* por su óptima relación rendimiento-coste y su capacidad para ejecutar el modelo *Llama 4 Maverick* con baja latencia.
- **Modelos de embeddings:** Se analizaron diferentes opciones como *OpenAI* y *sentence-transformers*, eligiendo finalmente el modelo *all-MiniLM-L6-v2*, de *setence-transformers*, por su equilibrio entre calidad y rendimiento.
- **Plataformas de despliegue:** Se evaluaron opciones como *Hugging Face Spaces*, *OpenAI* y *Anthropic*, seleccionando la primera por su facilidad de integración, enfoque en aplicaciones de IA y posibilidad de despliegue gratuito con acceso público utilizando *Streamlit*.

5.1.5. Resultados obtenidos

Los principales resultados obtenidos en este paquete de trabajo fueron:

1. Una base de conocimiento completa y estructurada sobre lenguajes formales, gramáticas y autómatas, organizada en archivos *Markdown*.
2. Una visión clara de las fortalezas y limitaciones de las herramientas existentes.
3. Un diseño técnico preliminar del sistema *RAG*, con la selección de *Qdrant* como base de datos vectorial y el modelo de *embeddings sentence-transformers/all-MiniLM-L6-v2*.
4. La definición de la arquitectura general de Fluxo, basada en *Streamlit* para la interfaz de usuario, *GroqCloud* para la inferencia del modelo *Llama 4 Maverick* y *Llama 3.3-70b* y *Hugging Face Spaces* para el despliegue.

5.2. PT2 DISEÑO CONCEPTUAL DEL SISTEMA

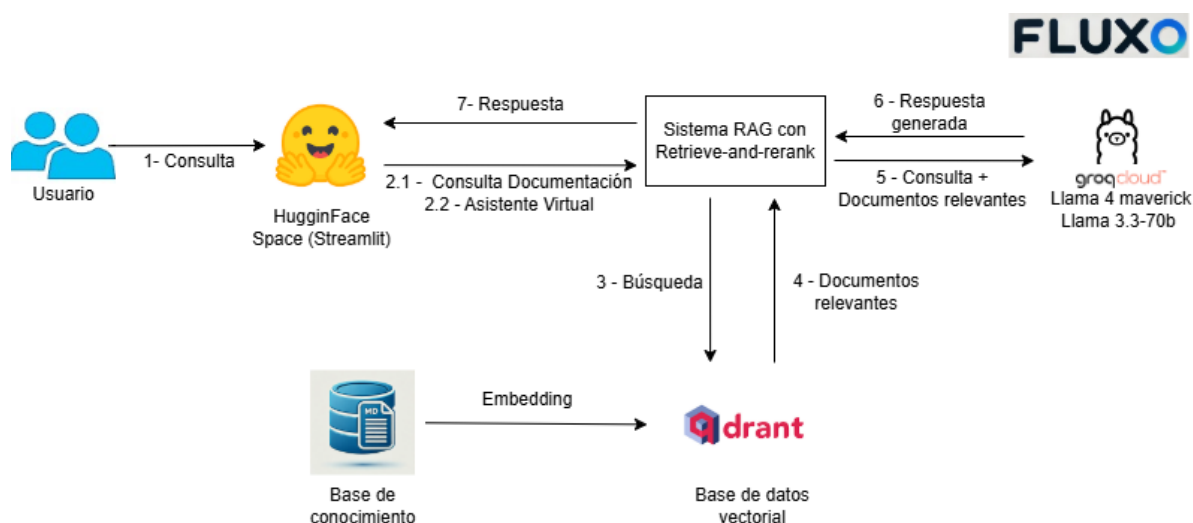


Figura 8: Diagrama de flujo

El segundo paquete de trabajo se centró en el diseño arquitectónico y funcional de Fluxo, estableciendo la estructura conceptual sobre la que posteriormente se desarrollaría la aplicación. Este diseño buscó responder a las necesidades identificadas en la fase de investigación previa, consolidando un enfoque técnico y pedagógico innovador.

5.2.1. Diseño del sistema RAG para gestión del conocimiento

Se desarrolló la arquitectura del sistema *RAG* como componente central, siguiendo el diagrama de flujo conceptual que se muestra en la imagen adjunta. Este sistema se diseñó para interconectar tres componentes fundamentales:

- **Interfaz de usuario:** Implementada como una aplicación web en *HuggingFace Space* mediante *Streamlit*.
- **Base de datos vectorial:** *Qdrant* para el almacenamiento y recuperación eficiente de *embeddings*.
- **Modelo generativo:** *Llama 4 Maverick* en *GroqCloud* para la generación de *feedback* en la resolución de ejercicios y *llama 3.3-70b versatile* para la consulta de documentación.

El flujo de operación se definió en siete pasos secuenciales, desde la consulta inicial del usuario hasta la entrega de la respuesta final:

1. El usuario introduce una consulta en la interfaz web.
2. La aplicación *Streamlit* procesa la petición según su tipo (consulta de documentación o asistente virtual).
3. El sistema *RAG* busca los documentos relevantes en *Qdrant*.

4. *Qdrant* devuelve los documentos con mayor similitud semántica y los reordena basándose en la arquitectura *Retrieve-and-rerank*.
5. El sistema *RAG* envía la consulta original junto con los documentos relevantes al modelo generativo.
6. *GroqCloud* procesa la información y genera una respuesta contextualizada.
7. La respuesta elaborada se presenta al usuario a través de la interfaz.

Esta arquitectura permitió facilitar el mantenimiento y garantizar la escalabilidad del sistema mediante la adición incremental de documentación sin necesidad de reentrenamiento.

5.2.2. Definición del formato estructurado para la base de conocimiento

Se estableció un formato estructurado para organizar la base de conocimiento, dividiéndola en dos categorías principales:

- **Documentación teórica:** Contenido formativo sobre lenguajes formales, gramáticas y autómatas, así como algoritmos prácticos.
- **Ejercicios resueltos:** Problemas tipo con soluciones detalladas.

Tras analizar diversas alternativas, se seleccionó *Markdown* como formato base debido a su legibilidad, facilidad de mantenimiento y compatibilidad con sistemas de control de versiones. La estructura de cada documento se diseñó jerárquicamente mediante encabezados, lo que permitiría posteriormente una segmentación semántica natural durante el proceso de vectorización.

Para los ejercicios, se adoptó un esquema "*Problem-Solution*" que separaba claramente el enunciado de la respuesta correcta, la cual se estructuraba paso a paso, facilitando así el análisis diferenciado por parte del *copilot* educativo y su posterior división en fragmentos. Este formato resultó especialmente adecuado para evaluar las soluciones propuestas por los usuarios, comparándolas con las soluciones de referencia.

5.2.3. Conceptualización de los módulos educativos

Se diseñaron dos módulos funcionales clave, cada uno con un propósito educativo específico:

- **Chatbot especializado:** Concebido para proporcionar acceso rápido y preciso a la documentación técnica. Se diseñó para responder a consultas puntuales sobre terminología, conceptos y fundamentos teóricos, utilizando un enfoque conversacional que facilitara el aprendizaje exploratorio.
- **Copilot educativo (Asistente Virtual):** Planteado como un asistente para la resolución de problemas, capaz de analizar paso a paso las soluciones propuestas por los estudiantes. Su diseño contempló la capacidad de identificar errores específicos, ofrecer retroalimentación personalizada y guiar hacia la comprensión profunda de los conceptos subyacentes.

Para cada módulo, se definieron plantillas de *prompts* específicas que estructurarían la comunicación con el modelo generativo, optimizando sus respuestas para cada caso de uso particular. Estas plantillas contemplaron instrucciones precisas sobre el formato, tono y profundidad de las respuestas, así como reglas para la expansión de abreviaturas técnicas.

5.2.4. Diseño preliminar de la interfaz de usuario

Se elaboró un boceto conceptual de la interfaz de usuario, priorizando la simplicidad, accesibilidad y usabilidad. El diseño contempló:

- Navegación por pestañas: Permitiendo alternar fácilmente entre el *chatbot* y el *copilot* educativo.
- Áreas de entrada de texto: Campos específicos para consultas, enunciados de problemas y soluciones propuestas.
- Herramientas auxiliares: Funcionalidades para facilitar la escritura, como un generador de tablas y un glosario de símbolos matemáticos.
- Panel lateral informativo: Con datos sobre el proyecto, autores y estado del desarrollo.

Este diseño preliminar sentó las bases para la posterior implementación en *Streamlit*, garantizando una experiencia de usuario intuitiva y eficiente.

5.2.5. Resultados y desafíos del paquete

Al finalizar este paquete de trabajo, se obtuvo un diseño arquitectónico completo del sistema RAG, una estructuración clara del formato para la base de conocimiento, una definición funcional de los módulos educativos y un esquema preliminar de la interfaz de usuario.

Entre los principales desafíos enfrentados destacaron:

- La definición de una arquitectura que permitiera escalabilidad sin comprometer el rendimiento
- La estructuración de la base de conocimiento para optimizar la recuperación semántica
- El diseño de *prompts* que maximizaran la precisión y relevancia de las respuestas generadas

5.3. PT3 DESARROLLO TÉCNICO

El desarrollo técnico de Fluxo se estructuró en torno a la implementación integral del sistema RAG, abarcando desde la preparación de la base de conocimiento hasta el despliegue multiplataforma de la aplicación web. El trabajo se dividió en cuatro etapas principales, cada una de las cuales requirió la integración de distintos módulos del *backend* y la coordinación de diferentes tecnologías.

5.3.1. Creación y estructuración de la base de conocimiento en *Markdown*

Se llevó a cabo la elaboración y organización de la base de conocimiento mediante espacios especializados en *Perplexity*, a los cuales se adjuntaron libros y documentos PDF obtenidos en el PT1, así como indicaciones claras del formato que debían seguir. Esta metodología permitió generar eficientemente el corpus inicial de documentación.

Los contenidos se estructuraron en archivos *Markdown* siguiendo una jerarquía clara de encabezados, permitiendo una segmentación semántica precisa. Cada documento se dividió en secciones temáticas y, en el caso de los ejercicios, se adoptó el formato *Problem-Solution* para facilitar su posterior análisis y recuperación.

Tras varias pruebas, se observó que el modelo ofrecía respuestas significativamente más precisas cuando en cada consulta se apoyaba en el algoritmo pertinente para la resolución del problema. En consecuencia, se desarrollaron todos los algoritmos requeridos para resolver los problemas del *dataset*, enriqueciendo así la base de conocimiento con procedimientos paso a paso, y logrando mayor granularidad a la hora de analizar las soluciones.

También se desarrollaron los algoritmos necesarios para la resolución de los problemas, se incluyeron en el *Template* para que sirviera de apoyo al modelo a la hora de localizar errores.

5.3.2. Conversión de los archivos *Markdown* a *embeddings* vectoriales y almacenamiento en *Qdrant*

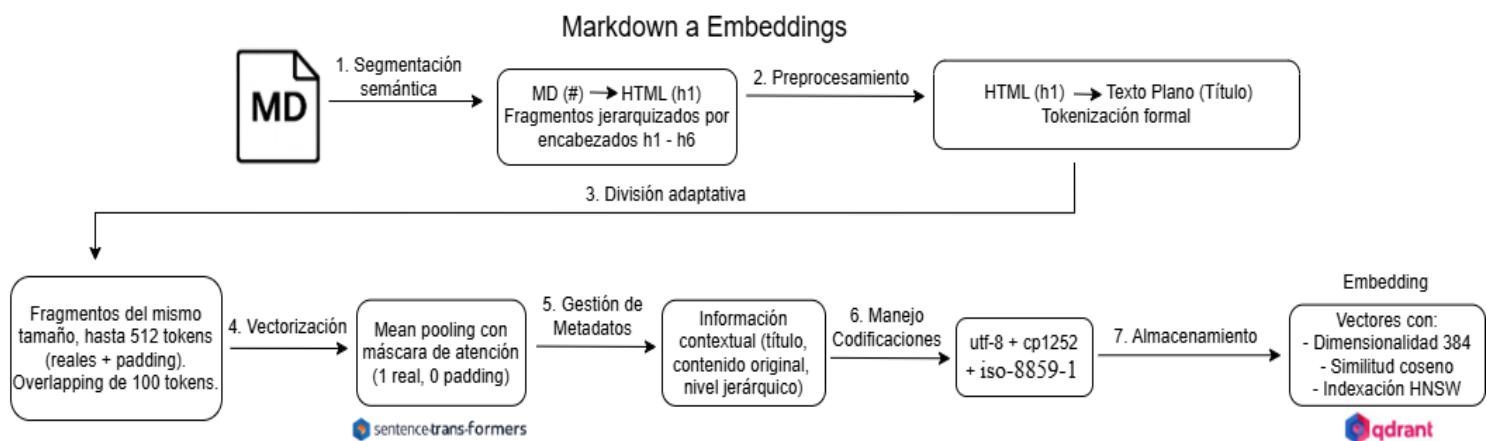


Figura 9: Diagrama de flujo de conversión MD a Embedding.

Una vez recopilada la documentación, se diseñó un *pipeline* (*md2vector.py*) para transformar los archivos *Markdown* en *embeddings* vectoriales mediante las siguientes etapas:

1. Segmentación semántica: Los documentos se segmentaron estructuralmente mediante su conversión a *HTML*, donde los encabezados (h1-h5) funcionaron como

delimitadores naturales. Este enfoque preservó la relación lógica entre títulos y contenidos, evitando dejar frases sin contexto.

2. Preprocesamiento del texto: Posteriormente se transformó de *HTML* a texto plano eliminando etiquetas, pero conservando la disposición lógica del contenido, y se realizó una tokenización formal de los *chunks*.
3. División adaptativa por *tokens*: se dividieron por fragmentos de hasta 512 *tokens* (reales + *padding*), con ajuste dinámico adaptativo para no superar el límite. Por otro lado, se introdujo *overlapping* de 100 *tokens* para no perder información de *tokens* calificados de *padding*.
4. Vectorización mediante *mean pooling*:
 - Se obtuvo un documento dividido en fragmentos del mismo tamaño; los *tokens* que componen cada fragmento tienen un vector oculto asociado que es la salida de la capa *embedding* del *transformer*.
 - Para convertir estos múltiples vectores en una representación única del fragmento completo, se realiza un promedio ponderado:

Se suma cada vector oculto (*hi*) multiplicado por su máscara de atención (*mi*), y se divide por el total de *tokens* reales. Esta operación matemática genera un vector único de 384 dimensiones que preserva las características semánticas de todo el fragmento de texto, independientemente de su longitud.
5. Gestión de metadatos: Los *embeddings*, que son estas representaciones vectoriales densas de 384 dimensiones, se almacenaron junto con información contextual, (título, contenido original, nivel jerárquico) facilitando la recuperación precisa.
6. Manejo de codificaciones: Se implementaron estrategias de *fallback* (*utf-8*, *cp1252*, *iso-8859-1*) para garantizar la correcta interpretación de caracteres especiales en todos los documentos.
7. Implementación en *Qdrant* (*collections_Qdrant.py*)

La base de datos vectorial se estructuró en dos colecciones especializadas:

1. *documentation_vectors*: *embeddings* de la documentación teórica.
2. *exercise_vectors*: *embeddings* de los ejercicios resueltos y sus soluciones.

Cada colección se configuró con parámetros optimizados:

- Dimensionalidad: 384 dimensiones (coincidente con la salida del modelo).
- Métrica de similitud: Coseno, elegida por su eficacia para comparar orientación vectorial independientemente de la magnitud, y de esta forma capturar la relación semántica.
- Indexación: *KNN* optimizada por su variante *HNSW* para búsquedas aproximadas de vecinos cercanos, con *ef_construct=128*, parámetro que controla cuántos vecinos se consideran durante la construcción del índice. Un valor más alto implica que el algoritmo buscará de forma más exhaustiva los vecinos más cercanos al añadir nuevos puntos al índice, lo que mejora la precisión del índice final, pero

también incrementa el tiempo y los recursos necesarios para construirlo. Por otro lado, *ef_search*=64, parámetro se utiliza durante las búsquedas (consultas) y determina cuántos vecinos se evalúan al buscar los puntos más cercanos en el índice. Un valor más alto aumenta la precisión de las búsquedas, ya que el algoritmo explora más opciones, pero también puede hacer que las búsquedas sean más lentas.

- Carga eficiente: Procesamiento por lotes de 16 elementos para optimizar memoria y velocidad.

Resultados del proceso

El sistema transformó más de 300 archivos *Markdown* en aproximadamente 2.350 fragmentos vectorizados. Esta implementación logró:

- Búsquedas ultrarrápidas: Tiempos de respuesta inferiores a 250ms.
- Alta precisión semántica: Recuperación de fragmentos contextualmente relevantes incluso con variaciones en la formulación de consultas.
- Escalabilidad: Capacidad para incorporar nuevos documentos sin reentrenamiento del modelo.
- Reducción del contexto: Selección eficiente de solo los fragmentos más relevantes, optimizando el uso de tokens en las consultas al modelo generativo.

5.3.3. Implementación del sistema RAG integrando recuperación semántica y generación de respuestas

El núcleo funcional del sistema (*chatbot.py*) se basó en la integración de tres módulos principales:

- Módulo de recuperación semántica: Ante cada consulta, el sistema generaba un *embedding* de la consulta con el mismo modelo *sentence-transformers/all-MiniLM-L6-v2* y lo utilizaba para buscar en *Qdrant* los fragmentos más similares de la base de conocimiento, empleando la distancia coseno como métrica de similitud y limitando el resultado a los 3 documentos más cercanos, es decir, aquellos cuya distancia era próxima a 1. Este proceso garantizaba que el modelo generativo dispusiera siempre del contexto más pertinente para la generación de respuestas.

Tras la recuperación inicial de los tres fragmentos más similares, se introdujo una etapa adicional de *reranking* neural utilizando el modelo *cross-encoder/ms-marco-MiniLM-L-6-v2*. Este modelo reevaluó la relevancia contextual entre la consulta del usuario y cada uno de los fragmentos recuperados, analizando interacciones profundas a nivel de *tokens* mediante atención bidireccional y procesando ambos textos concatenados, permitiendo detectar matices semánticos como equivalencias entre términos técnicos, negaciones implícitas o dependencias contextuales complejas.

La implementación de esta capa añadió un incremento controlado de latencia (180-220 ms adicionales), derivado del coste computacional de ejecutar inferencias neurales individuales para cada par consulta-fragmento.

- **Módulo generativo con GroqCloud:** Los documentos recuperados, junto con la consulta original, se estructuraron en un *prompt* específico y se enviaron al modelo *Llama 4 Maverick* alojado en *GroqCloud*, en el caso del *copilot*, ya que para el *chatbot* se empleó el modelo *llama 3.3-70b versatile*, que tenía la capacidad suficiente para abordar el cometido de consulta de documentación, de esta forma se ganó en capacidad de la herramienta, al tener cada modelo limitaciones independientes, por tanto las funcionalidades podrían funcionar en paralelo sin pisarse la una a la otra. Las plantillas de *prompt* resultaron fundamentales para el funcionamiento óptimo del sistema, diseñándose con una arquitectura cuidadosamente estratificada: primero definían el rol del asistente ("experto en gramáticas y autómatas"), luego incorporaban el contexto más relevante junto con el algoritmo de apoyo y finalmente proporcionaban instrucciones precisas de procesamiento, análisis y respuesta.

El *chatbot* (*TEMPLATE_ANSWER*) incluyó instrucciones críticas de como estructurar la respuesta, como la expansión sistemática e identificación de abreviaturas técnicas (como "GIC" a "Gramática Independiente de Contexto").

Mientras que para el *copilot* (*TEMPLATE_PROBLEM*) se construyó una plantilla especializada en evaluación paso a paso, con directrices específicas para analizar la solución del usuario frente al contexto recuperado.

Esta ingeniería de *prompts* permitió controlar con precisión el comportamiento del modelo generativo, asegurando respuestas pedagógicamente valiosas y técnicamente precisas.

- **Gestión avanzada de contexto:** Se implementó para el *chatbot* un mecanismo específico para analizar la dependencia contextual entre la pregunta actual y la inmediatamente anterior, permitiendo resolver referencias simples sin exceder los límites de *tokens* en consultas sucesivas. Este sistema funcionaba de la siguiente manera:
 - **Almacenamiento limitado:** El sistema guardaba únicamente la consulta inmediatamente anterior para evitar acumulación de *tokens* que pudiera superar los límites del modelo.
 - **Análisis de ambigüedad:** Mediante la plantilla de *prompt* especializada (*TEMPLATE_CONTEXT_ANALYSIS*), se enviaba al modelo en *Groq* la pregunta actual junto con la anterior, solicitándole que determinara si existían referencias ambiguas como pronombres o términos como "este" o "eso".
 - **Reformulación mínima:** Si se detectaban referencias ambiguas, el sistema incorporaba solo el contexto esencial de la pregunta anterior para clarificar la consulta, manteniendo la eficiencia en el uso de *tokens*.
 - **Preservación semántica:** Este enfoque permitía mantener la coherencia conversacional básica sin necesidad de almacenar o procesar el historial completo de la conversación, logrando un equilibrio entre contexto y eficiencia.

5.3.4. Desarrollo de la aplicación web con *Streamlit* y despliegue en *Hugging Face Spaces*



Figura 10: Funcionalidad de consulta de documentación.

La interfaz de usuario se desarrolló utilizando *Streamlit* (***app.py***), permitiendo la creación de una aplicación web interactiva, accesible y multiplataforma. Se diseñó una navegación por pestañas que diferenciaba claramente entre el módulo de consulta de documentación y el asistente virtual para resolución de problemas. Entre las funcionalidades implementadas destacaron:

- Áreas de entrada de texto para consultas, enunciados de problemas y soluciones propuestas.
- Herramientas auxiliares como un generador de tablas y un glosario de símbolos matemáticos.
- Panel lateral informativo con detalles del proyecto.
- Manual de uso de Fluxo.

 **FLUXO**

Seleccione el tipo de consulta:
Asistente Virtual

 **Herramientas de escritura**

Glosario de Símbolos Matemáticos

Generador de Tablas

Filas: 3, Columns: 3

¿Dónde insertar la tabla?

Insertar en Enunciado, Insertar en Solución

Asistente Virtual

Ingrese el enunciado del problema:

Ingrese su solución completa:

Evaluar Solución

Figura 11: Funcionalidad de asistente virtual.

El despliegue final se realizó en *Hugging Face Spaces*, lo que implicó implementar un flujo de trabajo *Git* para gestionar el código de la aplicación. El proceso consistió en crear un repositorio específico para el *Space*, configurar los controles de acceso mediante *tokens* de autenticación y establecer un sistema de despliegue continuo.

Para la implementación, fue necesario generar un archivo *requirements.txt* con dependencias específicas, incluyendo el *framework Streamlit* y la versión exacta de las bibliotecas, así como los clientes de *Qdrant* y *Groq* (*API Keys* + *URL*). La aplicación se configuró con mecanismos de caché para optimizar el rendimiento, especialmente crítico para la persistencia del procesador *RAG* entre sesiones de usuario.

5.4. PT4 PRUEBAS Y VALIDACIÓN

Descripción del proceso

El paquete se estructuró para garantizar que Fluxo cumpliera con los objetivos planteados, mediante la aplicación de pruebas exhaustivas y una evaluación continua a lo largo del desarrollo. Las actividades se centraron en cuatro ejes principales:

Validación de la base de conocimiento

La veracidad y corrección de la base de conocimiento se validó de forma exhaustiva. El tutor del TFG revisó uno a uno todos los problemas incluidos en la base de datos, hasta donde le dio tiempo, registrando los errores encontrados en un archivo denominado Revisión. Este archivo sirvió como referencia para depurar los ejercicios y como punto de partida para las pruebas funcionales posteriores.

Pruebas funcionales del *chatbot* y *copilot* educativo

Para verificar su precisión, se utilizaron los ejercicios que presentaban errores, contrastando la capacidad de Fluxo para detectarlos con la información recogida en el archivo Revisión. Además, se realizaron comparativas con IAs de pago como *Perplexity*, alimentando ambos sistemas con los mismos ejercicios erróneos y anotando los errores detectados por cada uno. Este proceso permitió construir un *benchmark* de evaluación humana y matemática, reflejado en un informe de resultados.

Se seleccionó una muestra representativa de 15 ejercicios sobre gramáticas y lenguajes, abarcando los diferentes tipos de problemas presentes en la base de datos original. En el área de autómatas y máquinas de Turing, no se pudo completar la revisión a tiempo, por lo que las pruebas se centraron en los ejercicios de gramáticas y lenguajes formales, logrando una visión representativa del funcionamiento de Fluxo.

La siguiente tabla resume los resultados obtenidos en la muestra analizada:

Ejercicio	Flujo	IA de pago
53: Construcción de una GR equivalente a una ER	✓ Ambos identifican correctamente que la gramática 2 no es regular y la producción que la invalida.	✓
57: Cardinalidad de un lenguaje	✓ El hecho de que tenga dos niveles de derivación y un único símbolo no terminal, implica que en el nivel cero y en el nivel 1 aparece repetido el mismo y único no terminal (para poder hacer el segundo nivel del árbol al desarrollar ese símbolo no terminal). Es decir, la gramática es recursiva, porque tiene una regla recursiva, donde se repite el no terminal en el lado izquierdo y derecho de la regla. Como es recursiva, su lenguaje es infinito.	✗
58: Construcción de una GIC para un lenguaje	✓ Ambos identifican correctamente que no satisface el lema de bombeo y por tanto es dependiente de contexto.	✓
59: Eliminar la recursividad por la izquierda	✓ El problema resuelve bien eliminar la recursividad por la izquierda en un paso, pero no resuelve la recursividad por la izquierda en varios pasos, es decir, elimina la directa pero no la indirecta.	✗ Rara vez lo detecta
60: Construcción de una gramática sin recursividad y en FNC	✓ Ambos identifican que la gramática final no está en FNC y la producción que la invalida.	✓
62: Determinar si una cadena pertenece a un lenguaje usando CYK	✓ Ambos detectan que la gramática inicial no está en FNC y la producción en la que se invalida ($C \rightarrow cA$), no se puede aplicar CYK, y por tanto hay problemas derivados a raíz de eso.	✓
63: Construcción de una gramática en FNG equivalente a otra G.	✓ Ambos detectan que faltan pasos intermedios en la explicación de la solución.	✓
65: Construcción de una GR equivalente a una ER	✓ Las reglas no cumplen con la forma de una gramática regular: $S \rightarrow T U$ es una producción unitaria. $T \rightarrow aT \mid bT \mid V$: Incorrecto porque; $T \rightarrow V$ también es una producción unitaria.	✓
66: Determinar si una cadena pertenece a un lenguaje usando CYK	✓ Ambos detectan que la gramática inicial no está en FNC y la producción en la que se invalida ($C \rightarrow B$), no se puede aplicar CYK, y por tanto hay problemas derivados a raíz de eso.	✓

67: Simplificación de una GIC	✗ Utiliza mal el algoritmo 1 de limpieza. Si lo usara bien, se daría cuenta que: 1. A es un símbolo generativo 2. En la segunda iteración del algoritmo, S pasa a ser un símbolo generativo 3. El algoritmo termina, ni B ni C son símbolos generativos, así que se eliminan todas las reglas donde aparecen. Por tanto, la solución original falló en no eliminar todas las reglas que contienen símbolos no generativos.	✗
68: Construcción de una GR usando el método de las derivadas.	✓ La solución proporcionada no aborda el problema planteado, aunque genera una GR correcta. Se centra en simplificar una gramática que no se deriva de la expresión regular dada. Para resolver el problema original, se debería aplicar el algoritmo adecuado para convertir la expresión regular $\alpha = ab^*(a + ab^*)$ en una gramática regular.	✗
69.1: Simplificación de una GIC.	✗ El paso 1.3 no está bien, el único símbolo anulable es S, pero S no aparece en la parte derecha de ninguna regla así que solo eliminamos la regla $S \rightarrow \lambda$ y no aparece ninguna regla nueva. Por tanto, habría que eliminar solo $S \rightarrow \lambda$, sin añadir producciones falsas	✓ A veces lo detecta mal
70: Construcción de una gramática en FNG para un lenguaje	✗ La solución original es incorrecta porque no garantiza $m \geq n$, es decir, que el número de "b"s sea siempre mayor o igual al de "a"s, cumpliendo con la definición de L. Por tanto genera todas las cadenas de L y muchas más, así que no es la solución buscada.	✓ A veces lo detecta mal
71: Determinar si una cadena pertenece a un lenguaje usando CYK	✓ Ambos detectan que la gramática inicial no está en FNC y la producción en la que se invalida ($S \rightarrow bC$), no se puede aplicar CYK, y por tanto hay problemas derivados a raíz de eso.	✓
72: Construcción de una gramática equivalente a otra en FNG	✓ La producción $Y_1 \rightarrow C C C$ debería transformarse adicionalmente para cumplir con FNG. Una posible corrección sería definir $Y_1 \rightarrow X_c Y_3$, $Y_3 \rightarrow C Y_4$, $Y_4 \rightarrow C C$, asegurando que cada paso comience con un terminal o sea válida en FNG.	✓

Tabla 2: Resultados

Fluxo logró identificar correctamente el error en 12 de los 15 ejercicios, al igual que la IA de pago, aunque en 3 de ellos muchas veces no era capaz de identificarlo (puntuación ponderada de 10,5/15). Con esto se observó que Fluxo siempre detecta el error, explicado de una forma más o menos precisa, pero siempre lo detecta, mientras que la IA de pago en varias ocasiones no era capaz de detectarlo, por ello se penalizó la ponderación respectiva a la IA de pago.

Esto evidenció que el sistema alcanzó un nivel de precisión comparable a soluciones comerciales de referencia, específicamente en la detección de errores conceptuales y estructurales en las soluciones de gramáticas y lenguajes formales.

Evaluación del rendimiento del sistema *RAG*

El rendimiento del sistema *RAG* se evaluó en términos de tiempos de respuesta y relevancia de la información recuperada. Para ello, se emplearon los sistemas de observabilidad y control proporcionados por *Qdrant* y *GroqCloud*, que ofrecieron paneles de monitorización en tiempo real sobre el uso de recursos, latencia y disponibilidad del servicio.



Figura 12: Dashboard Groq.

La parte superior ilustra los códigos de estado *HTTP* registrados durante las pruebas, permitiendo identificar la frecuencia y el tipo de respuestas del servidor (por ejemplo, éxitos o posibles errores), lo que facilita la detección de incidencias en la comunicación entre componentes.

En la parte inferior se observa el seguimiento de las peticiones realizadas al modelo *llama-4-maverick-17b-128e-instruct*, así como el número total de *tokens* procesados (entrada y salida). Esto permite comprobar que el sistema no ha superado los límites establecidos y que el volumen de consultas se mantiene dentro de los parámetros esperados. También permite medir la cantidad de tokens que le pasamos entre el *Template*, la consulta y los archivos recuperados, midiendo si conviene comprimirlo.

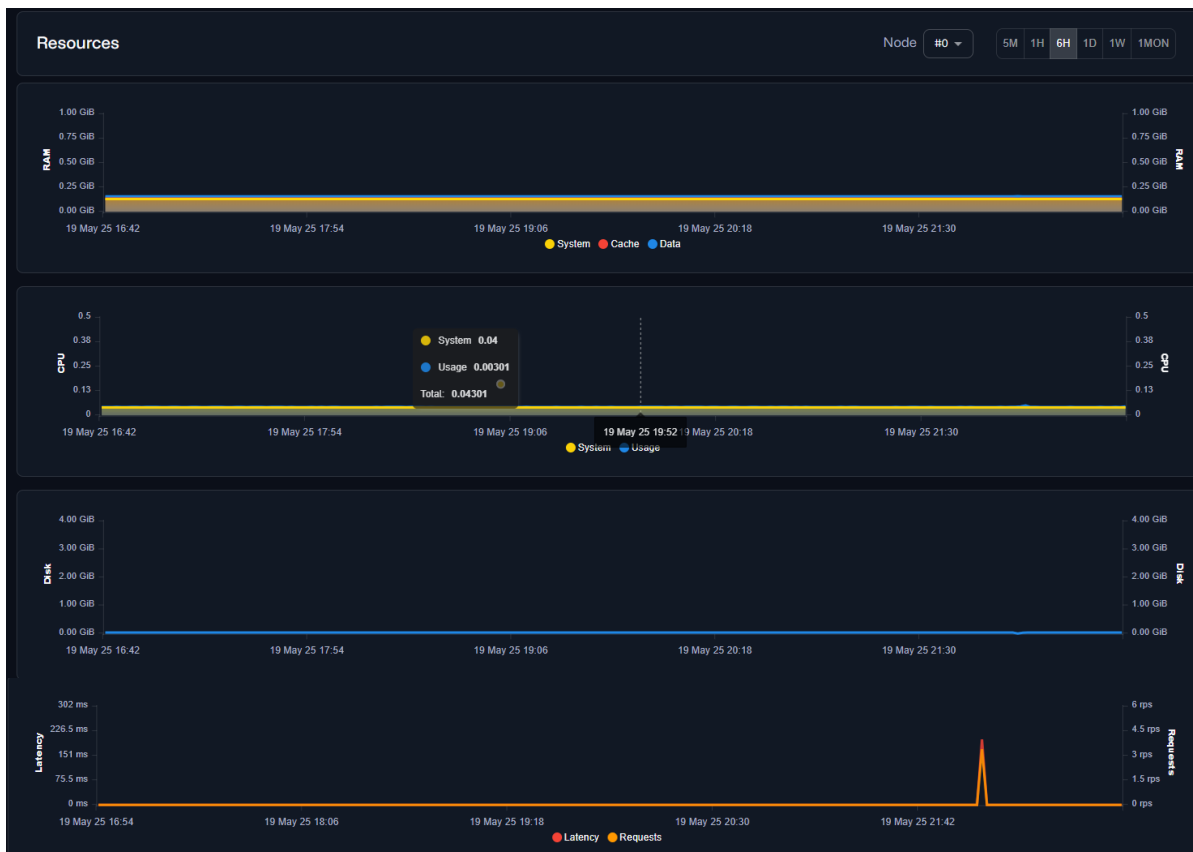


Figura 13: Dashboard Qdrant.

Esta figura muestra los gráficos de uso de recursos del sistema, incluyendo memoria *RAM*, *CPU* y disco. Estos datos permiten verificar que el consumo de recursos es estable y bajo, sin picos que puedan indicar problemas de rendimiento o cuellos de botella.

La monitorización continua de estos parámetros resulta esencial para garantizar la disponibilidad y la escalabilidad del servicio, ya que permite anticipar posibles caídas o degradaciones y tomar medidas proactivas, así como identificar si conviene aumentar la capacidad pasando a una versión de pago.

Los tiempos de respuesta medios se situaron entre 1 y 6 segundos por consulta, dependiendo de la complejidad, con una latencia de búsqueda en *Qdrant* inferior a 250 ms y una generación de respuestas por parte del modelo *llama 4 Maverick* de entre 1,2 y 4 segundos.

La relevancia de la información recuperada se validó incluyendo en la respuesta de Fluxo el nombre y orden de los archivos fuente, utilizando *retrieve-and-rerank* para priorizar los fragmentos más pertinentes.

Validación supervisada y utilidad educativa

La validación supervisada de la calidad de la retroalimentación y la utilidad de Fluxo como herramienta educativa se realizó de manera limitada, ya que solo se podrá comprobar una vez los alumnos comiencen a utilizar la herramienta y aportar *feedback*, así como la capacidad de la aplicación para soportar el tráfico de múltiples usuarios simultáneos. No obstante, las primeras pruebas con usuarios externos y docentes confirmaron que la retroalimentación

proporcionada por el *copilot* era detallada, clara y útil para guiar el aprendizaje activo y la comprensión profunda de los conceptos.

5.5. PT5 DOCUMENTACIÓN FINAL

La documentación del proyecto se estructuró como un artefacto técnico-pedagógico integral, cumpliendo un triple propósito: registro del desarrollo realizado, guía de uso para usuarios finales y base para futuras mejoras. El proceso de redacción siguió un enfoque sistemático, alineado con los estándares de ingeniería de software y buenas prácticas en investigación educativa.

5.5.1. Estructura y contenidos de la memoria final

1. Antecedentes y fundamentación teórica

- Análisis crítico del estado del arte en enseñanza de lenguajes formales.
- Justificación técnica de la arquitectura *RAG* frente a alternativas como *fine-tuning* de modelos.
- Estudio comparativo de herramientas existentes (*JFLAP*, *Automata Tutor*, etc) y sus limitaciones.

2. Diseño metodológico

- Diagrama detallado del flujo *RAG* con explicación técnica de cada componente.
- Protocolos de desarrollo iterativo aplicados (*sprints* mensuales con entregables definidos).
- Criterios de selección tecnológica (*benchmarks* de rendimiento, análisis coste-beneficio).

3. Implementación técnica

- Especificaciones de la base de conocimiento:
 - Estructura jerárquica de archivos *Markdown*.
 - Metodología de segmentación semántica (niveles h1-h6).
 - Estadísticas finales (384 dimensiones por embedding).
- Configuración de infraestructura:
 - Parámetros de *Qdrant* (*HNSW*).
 - Optimizaciones en *Hugging Face Spaces* (caché, gestión de memoria).

4. Resultados y validación

- Tablas comparativas de rendimiento (tiempos de respuesta, precisión semántica).

- Análisis exhaustivo de las pruebas con usuarios
- Análisis técnico de los 15 casos límite identificados durante las pruebas.

5. Anexos técnicos

- Manual de instalación, uso y configuración avanzada.
- Ejemplos prácticos de uso con capturas de pantalla comentadas.

5.5.2. Proceso de elaboración

La redacción siguió un enfoque incremental, integrando:

- Versionado estricto: Uso de *Git* con ramas específicas para cada sección (teoría, metodología, resultados).
- Revisiones iterativas: 4 ciclos de *feedback* con el tutor del TFG, centrados en claridad técnica y rigor académico.
- Validación cruzada: Contrastación de los datos técnicos con los logs del sistema y resultados de pruebas.
- Normas de estilo: Aplicación de estándares *IEEE* para documentos técnicos y pautas *APA* para referencias bibliográficas.

6. RESULTADOS

Benchmark comparativo:

Herramienta	Precisión	Tiempo respuesta	Feedback granular
Fluxo	80% - Alcance limitado	5s	Sí
Perplexity	70%	4s	Sí
Automata Tutor	100% - IA tradicional	N/A	Sí
JFLAP	N/A	N/A	No

Tabla 3: Benchmark

Dificultades y desviaciones respecto a lo previsto

- El objetivo inicial fue integrar inteligencia artificial en *JFLAP* para potenciar la enseñanza de lenguajes formales y autómatas, mejorando la retroalimentación pedagógica y la resolución guiada de problemas. Sin embargo, este enfoque no resultó viable debido a las limitaciones técnicas y la rudimentariedad de su arquitectura.
- Posteriormente, se optó por crear una versión en *Java* con integración directa a *JFLAP*, pero tampoco resultó viable debido a la imposibilidad de convertirla en un *.jar* descargable debido a la incompatibilidad de las librerías de *Qdrant* y *jflap-lib*.



Figura 14: Versión java

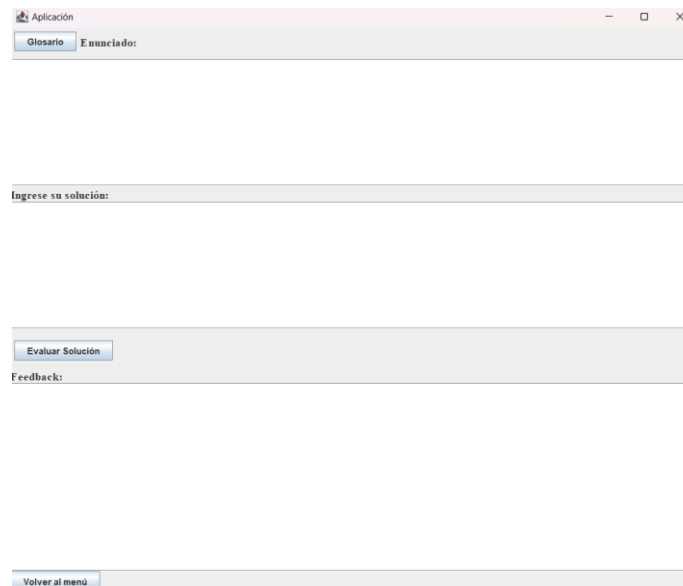


Figura 15: Funcionalidad de asistente virtual en java.

- No se pudo completar la revisión y validación de todos los ejercicios relativos a autómatas y máquinas de Turing, lo que limitó el alcance de las pruebas funcionales a los ejercicios de gramáticas y lenguajes
- Se optimizaron los *prompts* y la gestión del contexto para ajustarse a los límites de tokens impuestos por *GroqCloud*, limitando el potencial de Fluxo.

Resultados obtenidos y validación de objetivos

- Base de conocimiento y consulta de documentación: Se construyó una base de conocimiento completa y estructurada en archivos *Markdown*, abarcando teoría y ejercicios resueltos sobre lenguajes formales, gramáticas y autómatas, que incluían 235 ejercicios, unos 80 documentos teóricos y 16 algoritmos. Esta documentación se almacenó en la base de datos vectorial *Qdrant* y permitió la recuperación eficiente de información relevante mediante técnicas *RAG* con *Retrieve-and-rerank*. Se validó que el sistema recuperaba documentos semánticamente relevantes y que la extracción de información era precisa, ya que en las respuestas se adjuntaba un enlace directo al archivo fuente, permitiendo rastrear y verificar la información utilizada.
- Desarrollo y despliegue de la aplicación web: Se comprobó el correcto funcionamiento multiplataforma, ya que la aplicación fue accesible desde diferentes navegadores y dispositivos, sin requerir instalación local. Pruebas con usuarios externos confirmaron la estabilidad y la experiencia de usuario fluida, con tiempos de respuesta adecuados.
- Optimización y rendimiento del sistema *RAG*: Se midieron los tiempos de respuesta del sistema completo, verificando que eran adecuados para una interacción fluida, incluso con consultas complejas. El sistema demostró tiempos de respuesta bajos y alta relevancia semántica en la recuperación de información.

- El objetivo de garantizar la accesibilidad y usabilidad de la aplicación se cumplió plenamente. La aplicación fue acompañada de una guía de uso detallada, incluida en el anexo final, y se verificó su accesibilidad tanto en dispositivos móviles como de escritorio, permitiendo un acceso universal y facilitando la adopción por parte de estudiantes y docentes en contextos diversos.
- Validación del *chatbot* y *copilot* educativo:
Se validó que las sugerencias ofrecidas por el *copilot* fueran claras, detalladas y orientadas a guiar al usuario hacia una comprensión más profunda del problema planteado. Se compararon las respuestas generadas por Fluxo con los errores detectados en el archivo de revisión y con los resultados de una IA de pago, constatando la calidad del *feedback* proporcionado.

Se validó que la información recuperada por el *chatbot* fuera relevante, precisa y estructurada según lo especificado, sin incurrir en redundancias que restaran claridad a la respuesta.

Se validó que Fluxo alcanzaba una precisión del 80% en la detección de errores en ejercicios de gramáticas y lenguajes, igualando o superando el rendimiento de IAs comerciales en los casos analizados.

En conclusión, los resultados obtenidos evidenciaron que la solución final basada en *Streamlit*, *GroqCloud* y *Qdrant*, desplegada en *Hugging Face Spaces*, permitió cumplir los objetivos de accesibilidad, usabilidad y funcionalidad definidos para el proyecto, a pesar de las desviaciones y dificultades encontradas durante el desarrollo.

7. IMPLICACIONES ÉTICAS E IMPACTO SOCIAL

En esta sección se analizaron las implicaciones éticas y sociales del proyecto Fluxo. Se partió de la premisa de que la integración de inteligencia artificial en el ámbito educativo conlleva responsabilidades adicionales en cuanto al respeto a los derechos fundamentales, la protección de datos y la transparencia algorítmica. El análisis se fundamentó en el marco ético-legal vigente y en la identificación de riesgos, con el objetivo de garantizar un desarrollo responsable y respetuoso con las personas y el entorno.

7.1. MARCO ÉTICO Y LEGAL

El marco ético y legal del proyecto se estructuró en torno a los siguientes principios y normativas:

- No violación de los derechos humanos: Se garantizó que la aplicación respetara la dignidad humana, los derechos fundamentales y las libertades individuales, evitando vulnerar la privacidad, la libertad de expresión o cualquier otro derecho reconocido internacionalmente. Todas las decisiones de desarrollo se sometieron a revisión ética continua y se buscó la máxima transparencia en la implementación.
- Protección de datos personales y privacidad: Se cumplió con el Reglamento General de Protección de Datos (RGPD) [19], asegurando que los datos personales de los usuarios se trataran de forma segura y confidencial, aplicando el principio de minimización y las recomendaciones de la Agencia Española de Protección de Datos.
- Transparencia algorítmica y explicabilidad: Se procuró que los sistemas de IA integrados fueran auditables y comprensibles, evitando el uso de “cajas negras” que dificultaran la comprensión del funcionamiento interno de los algoritmos, en línea con el artículo 1.3 del Código Ético de la ACM.
- Honestidad intelectual y rigor científico: Se rechazaron prácticas que pudieran distorsionar resultados u ocultar limitaciones técnicas, siguiendo el artículo 1.3 del Código Ético de la ACM. [20]
- Neutralidad tecnológica y acceso igualitario: Se promovió el acceso igualitario a la herramienta, evitando favorecer a colectivos con mayor poder tecnológico, y facilitando el acceso a estudiantes y docentes independientemente de su contexto socioeconómico. Artículo 1.4 del Código Ético de la ACM. [20]
- Responsabilidad legal: Se establecieron protocolos claros para definir responsabilidades en caso de fallos técnicos o mal uso de la tecnología, incluyendo planes de respuesta ante incidencias.
- Sostenibilidad medioambiental: Se minimizó el impacto ambiental asociado al desarrollo y operación de la aplicación, priorizando el uso eficiente de recursos y la reducción de la huella de carbono. Artículo 2.9 del Código Ético de la ACM. [20]
- Normativa sobre inteligencia artificial: Se tuvo en cuenta el Reglamento de IA 2024/1689 del Parlamento Europeo [21], clasificando el sistema como de riesgo limitado y aplicando las medidas de transparencia y supervisión humana requeridas.

7.2. VALOR DEL PROYECTO

El valor principal del proyecto radicó en democratizar el acceso a recursos educativos avanzados en teoría de lenguajes y autómatas, áreas tradicionalmente complejas y poco accesibles para estudiantes sin apoyo especializado. Fluxo facilitó el aprendizaje autónomo mediante la integración de un *chatbot* para consulta de documentación y un *copilot* que ofreció retroalimentación paso a paso, permitiendo a los usuarios identificar y corregir errores en sus

soluciones. Esto supuso un avance respecto a herramientas previas como *JFLAP*, que solo proporcionaban respuestas finales sin guía pedagógica.

Además, la aplicación contribuyó a la reducción de la brecha digital y lingüística, al ofrecer recursos accesibles en español y adaptados a diferentes niveles de conocimiento, y promovió la equidad educativa al estar disponible de manera abierta para toda la comunidad universitaria.

7.3. ALCANCE

El alcance del proyecto se dividió en dos grandes grupos:

- Directamente afectados: Estudiantes y docentes de asignaturas relacionadas con lenguajes formales, gramáticas y autómatas, que utilizaron la herramienta como apoyo en el aprendizaje y la docencia.
- Indirectamente afectados: Cualquier usuario final que se benefició de los avances en metodologías educativas basadas en IA, así como instituciones educativas interesadas en la integración de tecnologías similares en sus programas de estudio.

La naturaleza abierta e inclusiva del proyecto permitió que su impacto trascendiera el ámbito universitario, facilitando la transferencia de conocimiento y fomentando la colaboración entre diferentes comunidades académicas y profesionales.

7.4. RESPONSABILIDAD E IMPACTO

Riesgos técnicos:

Se identificaron y evaluaron los principales riesgos asociados al desarrollo y uso de la aplicación, incluyendo:

Riesgo	Descripción	Probabilidad	Impacto	Acción
R1	Errores en la retroalimentación del copilot	Media	Alto	Validación exhaustiva mediante pruebas y revisión docente
R2	Filtración de datos personales	Baja	Alto	Aplicación de medidas de seguridad y cifrado: incluyen cifrado de las comunicaciones (HTTPS/TLS), aislamiento de entornos y gestión segura de credenciales. De igual modo, tanto GroqCloud como Qdrant proporcionan mecanismos de protección de datos en tránsito y en reposo, así como controles de acceso y autenticación robustos
R3	Caídas del servicio o fallos de acceso	Media	Medio	Monitorización y redundancia en la infraestructura: sistemas de observabilidad y control que ofrecen tanto Qdrant como GroqCloud, los cuales

				disponen de dashboards de monitorización en tiempo real. Estos paneles permiten supervisar el estado de los servicios, el uso de recursos, la latencia de las consultas y la disponibilidad, facilitando la detección temprana de incidencias y la toma de decisiones proactiva ante posibles caídas o degradaciones del servicio. Además, la infraestructura de Hugging Face Spaces garantiza una alta disponibilidad y mecanismos de recuperación ante fallos, contribuyendo a la resiliencia global del sistema desplegado
R4	Sesgos en las respuestas de la IA	Media	Medio	Entrenamiento con datos diversos y revisión continua

Tabla 4: Registro de Riesgos

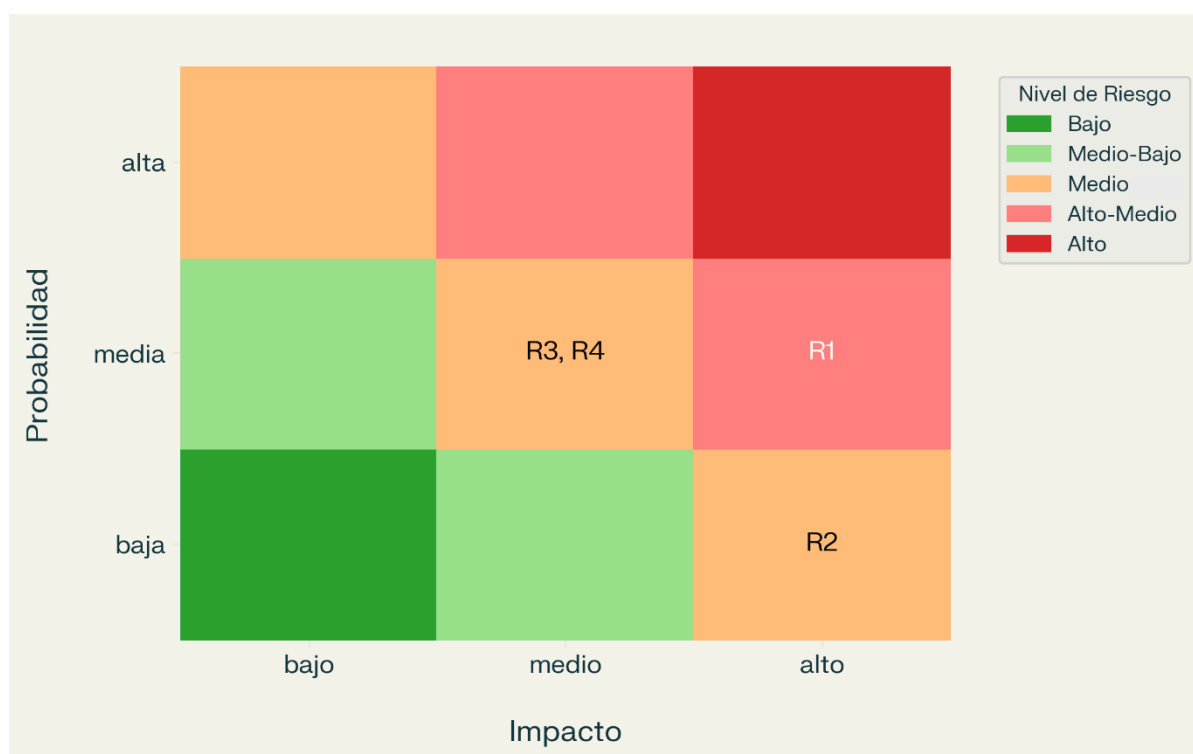


Figura 16: Matriz de Riesgos.

Impacto social:

El proyecto fomentó la democratización del acceso al conocimiento y la igualdad de oportunidades en el ámbito educativo. Sin embargo, se reconoció el riesgo de dependencia tecnológica y la posibilidad de que la herramienta no estuviera igualmente disponible para todos los colectivos si la infraestructura técnica no se mantuviera adecuadamente.

Impacto económico:

La integración de IA en la educación puede reducir costes asociados a tutorías presenciales y mejorar la eficiencia del proceso de aprendizaje. No obstante, se identificó el riesgo de que la inversión inicial en infraestructura y mantenimiento pudiera suponer una barrera para su adopción en entornos con recursos limitados.

Impacto medioambiental:

El uso de recursos en la nube y la computación intensiva puede aumentar el consumo energético. Se priorizó el uso eficiente de recursos y la selección de proveedores con políticas de sostenibilidad para minimizar el impacto ambiental.

7.5. CONCLUSIONES

En relación con todo lo expuesto, dado el valor que presentó el proyecto y que los riesgos detectados pudieron evitarse o se mantuvieron bajo control gracias a las medidas adoptadas, se concluyó que el proyecto no solo resultó viable desde el punto de vista ético, sino que además fue recomendable llevarlo a cabo. El análisis realizado permitió constatar que la integración de IA en la educación, cuando se realiza bajo un marco ético y legal riguroso, contribuye de manera significativa al bien común, promoviendo la equidad, la transparencia y el acceso universal al conocimiento.

8. CONCLUSIONES

Logros y aprendizajes clave

El desarrollo de Fluxo demostró el potencial transformador de la Inteligencia Artificial Generativa, validando que incluso con los servicios gratuitos de herramientas como *Groq* y *Qdrant* es posible crear sistemas robustos cuando se cuenta con una base de conocimiento estructurada y bien documentada. La experiencia reforzó tres convicciones fundamentales:

1. La viabilidad técnica de implementar asistentes especializados sin dependencia de *APIs* comerciales.
2. La importancia crítica de diseñar arquitecturas modulares que permitan evolucionar funcionalidades sin reestructuraciones complejas.
3. El valor estratégico de priorizar accesibilidad multiplataforma sobre integraciones nativas con *software legacy* como *JFLAP*.

A nivel personal, el proyecto me sirvió como catalizador profesional: consolidó habilidades técnicas en gestión de modelos *LLM*, diseño de sistemas *RAG* y despliegue de aplicaciones *cloud*, mientras reforzaba competencias transversales como la planificación ágil y la resolución creativa de obstáculos técnicos. La inmersión en tecnologías emergentes (desde *Streamlit* hasta *embeddings* vectoriales) confirmó mi interés por especializarme en desarrollo de soluciones *IA* aplicadas a casos reales.

Evolución futura

Mejoras técnicas:

- Modelos *premium*: Migrar a versiones avanzadas para aumentar capacidades de razonamiento lógico-matemático, esencial en problemas de autómatas.
- Clarificación interactiva: Implementar diálogos contextualizados que replanteen consultas ambiguas mediante plantillas dinámicas, reduciendo errores de interpretación, lo que también requeriría modelos *premium* al necesitar mayor capacidad de *tokens*.
- Integración *JFLAP*: Explorar emulación parcial de funcionalidades clave de *JFLAP* mediante bibliotecas Python, recuperando el objetivo inicial de integración desde un enfoque alternativo.

Ampliación funcional

- Cobertura de autómatas y máquinas de Turing: Desarrollar *pipelines* especializados para problemas no lineales, combinando representaciones gráficas con análisis simbólico.
- Sistema de *feedback* estructurado: Incorporar mecanismos automáticos de recolección y análisis de interacciones usuarias para priorizar mejoras basadas en datos reales.

Escalabilidad operativa

- Migración a infraestructura propia: Trasladar la aplicación a servidores propios para tener mayor control, permitiendo personalización avanzada de interfaz y funcionalidades.
- Modelo de negocio: Evaluar su conversión en servicio *SaaS* para instituciones educativas, ofreciendo planes *freemium* con soporte premium para personalizaciones.

Este proyecto sentó las bases técnicas y metodológicas para construir herramientas educativas *AI-driven* que complementen -no sustituyan- el rol docente. Su mayor valor radica en demostrar que, incluso con recursos limitados, es posible crear sistemas pedagógicos disruptivos cuando se combina conocimiento especializado con arquitecturas tecnológicas modernas. La experiencia acumulada servirá como punto de partida para iniciativas más ambiciosas en el ecosistema *EdTech*.

9. BIBLIOGRAFÍA

- [1] P. Linz and S. H. Rodger, An Introduction to Formal Languages and Automata, Seventh Edition, Jones & Bartlett Learning LLC, 2023.
- [2] Smith, M. D., Finley, J. H., & Professor, J. (2012). *In Praise of Engineering a Compiler*.
- [3] D. Hendrycks et al., “Measuring Mathematical Problem Solving With the MATH Dataset,” UC Berkeley, 2021. [Online]. Disponible: <https://arxiv.org/abs/2103.03874>
- [4] Reflex, “Top Python Web Development Frameworks in 2025,” Reflex Blog, 2024. [Online]. Disponible: <https://reflex.dev/blog/2024-12-20-python-comparison/>
- [5] Artificial Analysis, “Independent Analysis of AI Models and API Providers.” [Online]. Disponible: <https://artificialanalysis.ai/models/llama-4-maverick/providers>
- [6] Artificial Analysis, “Independent Analysis of AI Models and API Providers.” [Online]. Disponible: <https://artificialanalysis.ai/models/llama-4-maverick>
- [7] DataCamp, “The Top 5 Vector Databases.” [Online]. Disponible: https://www.datacamp.com/es/blog/the-top-5-vector-databases?dc_referrer=https%3A%2F%2Fwww.google.com%2F
- [8] GroqCloud Documentation, “GroqCloud Platform Overview.” [Online]. Disponible: <https://console.groq.com/docs/rate-limits>
- [9] Totalf Documentation, “Una herramienta para la enseñanza de lenguajes formales.” [Online]. Disponible: <https://bioinfo.uib.es/~joemiro/aenui/ProcSindi/P2005/1266.pdf>
- [10] Automata Tutor Documentation, “Automata Tutor: An Intelligent Tutoring System for Automata and Formal Languages.” [Online]. Disponible: <https://arxiv.org/pdf/2005.01419>
- [11] Khanmigo Evaluation, “An Evaluation of Khanmigo, a Generative AI Tool, as a Computer-Assisted Language Learning App.” [Online]. Disponible: <https://www.rnfinity.com/storage/uploads/6313.pdf>
- [12] Qdrant Documentation, “Vector Database for AI Applications.” [Online]. Disponible: <https://qdrant.tech/documentation>
- [13] Building a Retrieval-Augmented Generation (RAG) System for Academic Papers [Online]. Disponible: <https://cse.aua.am/files/2024/05/Building-a-Retrieval-Augmented-Generation-RAG-System-for-Academic-Papers.pdf>

- [14] Hugging Face, “Using GPU Spaces.” [Online]. Disponible: <https://huggingface.co/docs/hub/spaces-gpus>
- [15] Prompting Guide.” [Online]. Disponible: <https://console.groq.com/docs/prompting>
- [16] RAG Architectures. [Online]. Disponible: https://www.linkedin.com/posts/mart%C3%AD-domingo-51b769150_rag-ya-no-es-solo-retrieve-and-generate-activity-7328718371179442176-rAfo?utm_source=share&utm_medium=member_desktop&rcm=ACoAAE2CX_4BUkFBfVOPtuktB-XBR4MnB8xkE1Q
- [17] Documentation on RAG architectures. [Online]. Disponible: <https://arxiv.org/pdf/2005.11401>
- [18] Documentation on Retrieve-and-rerank RAG architectures. [Online]. Disponible: <https://aclanthology.org/2020.acl-main.228.pdf>
- [19] Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos y por el que se deroga la Directiva 95/46/CE (Reglamento general de protección de datos). [Online]. Disponible: <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX%3A32016R0679>
- [20] Código Ético de la ACM. [Online]. Disponible: <https://www.acm.org/about-acm/code-of-ethics-in-spanish>
- [21] Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo, de 13 de junio de 2024, por el que se establecen normas armonizadas en materia de inteligencia artificial. [Online]. Disponible: https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=OJ%3AL_202401689

MANUAL DE USO DE FLUXO: GUÍA PARA OBTENER RESPUESTAS COMPLETAS Y FIABLES

Este manual recoge las mejores prácticas para interactuar con Fluxo y maximizar la calidad, precisión y utilidad de las respuestas generadas por el asistente virtual, tanto en la consulta de documentación (*chatbot*) como en la resolución guiada de ejercicios (*copilot*). Se recogen directrices de *prompting* óptimo, formato de entrada, estructura de problemas y verificación, adaptadas a las características técnicas y pedagógicas de Fluxo.

1. Prompting óptimo en Fluxo

1.1. Principios fundamentales del prompting:

- El *prompt* es la especificación detallada que se proporciona al modelo. Cuanto más preciso y estructurado sea, mayor será la calidad y fiabilidad de la respuesta. [15]
- Un *prompt* óptimo debe contener cinco elementos: rol, instrucciones, contexto, input y output esperado.

El rol, las instrucciones y el output, no dependen del usuario, el input claramente sí, es lo que se introduce en los espacios de ENUNCIADO Y SOLUCIÓN.

Sin embargo, el contexto también depende del usuario, ya que en el input (ENUNCIADO) se puede especificar la contextualización SIEMPRE.

Por otro lado, está el output esperado, el cual es lo que esperamos que nos conteste Fluxo. Si esto no se cumple al completo, realizar un nuevo prompt especificándoselo en el ENUNCIADO.

1.2. Estructura de un prompt

Rol: “Eres un experto en gramáticas formales y autómatas finitos.”

Instrucciones: TEMPLATE_ANSWER Y TEMPLATE_PROBLEM.

Input: ENUNCIADO + SOLUCIÓN

Contexto: “Utiliza el método de subconjuntos para la conversión de AFND a AFD.”

Output esperado: “Incluye la verificación de la eliminación de la recursividad indirecta.”

1.3.Formato Markdown y presentación de tablas

Siempre emplear formato **Markdown** para estructurar la información, encabezados y tablas.

Los autómatas y máquinas de Turing deben representarse siempre en el siguiente formato de tabla:

El AFD se representa en la siguiente tabla:

Estado	Entrada 'a'	Entrada 'b'
q0	q0, q1	q0
q1	q2	-
q2	-	q2

Utilizar siempre símbolos matemáticos en **texto plano**, según el glosario de Fluxo.

Expresión	Texto Plano	Ejemplo de uso
cdot	·	$A \cdot B$
dot{x}	$\dot{x}$	$\dot{x}$
dots	...	$a_1, a_2, \dots, a_n$
cup	$\cup$	$A \cup B$
cap	$\cap$	$A \cap B$

1.4.Estructuración de la solución: pasos y verificación

Desglosar la solución en pasos **claramente identificables** con encabezados Markdown:

- # Paso 1
- ## 1.1
- # Paso 2
- etc.

Seguir los pasos de los **algoritmos** desarrollados en la base de conocimiento de Fluxo (se pueden consultar desde la funcionalidad de documentación).

Algoritmos ya implementados:

- Conversión de un Autómata Finito No Determinista (AFND) a un Autómata Finito Determinista (AFD) (Método de Subconjuntos).
- Conversión de un Autómata de Pila No Determinista (APND) en una Gramática Independiente del Contexto (GIC).
- Algoritmo Cocke-Younger-Kasami (CYK).
- Conversión de una Expresión Regular (ER) a Gramática Regular (GR) (Método de las derivadas).
- Conversión de una Expresión Regular (ER) a Gramática Regular (GR) a Autómata Finito (AF) (Método de las derivadas).
- Factorización a Izquierdas para Gramática Independiente del Contexto (GIC).
- Conversión de una Gramática Independiente de Contexto (GIC) a Forma Normal de Chomsky (FNC).
- Conversión de una Gramática Independiente de Contexto (GIC) a Forma Normal de Greibach (FNG).
- Conversión de una Gramática Independiente de Contexto (GIC) a un Autómata de Pila No Determinista (APND).
- Conversión de una Gramática Lineal por la Derecha (GLD) a Gramática Lineal por la Izquierda (GLI).
- Conversión de una Gramática Lineal por la Izquierda (GLI) a Gramática Lineal por la Derecha (GLD).
- Simplificación de una Gramática Independiente de Contexto (GIC) (eliminación de no generativos, inaccesibles, λ -producciones y unitarias).
- Minimización de un Autómata Finito Determinista (AFD).
- Eliminación de la Recursividad por la Izquierda en un paso.
- Eliminación de la Recursividad por la Izquierda en múltiples pasos (directa e indirecta).
- Algoritmo de Thompson: Conversión de Expresiones Regulares (ER) en Autómatas Finitos No Deterministas (AFND))

Incluir siempre un paso de **verificación** al final de la solución, comprobando explícitamente que:

Se han eliminado recursividades (directa e indirecta) si procede.

La gramática cumple las propiedades requeridas (GR, GIC, FNC, etc.).

No quedan producciones inútiles o inaccesibles.

El autómata o la gramática final cumple las condiciones del lenguaje.

1.5.Redacción de enunciado: estructura y especificidad

Estructurar siempre el enunciado en dos niveles:

- # Problema: Descripción genérica del tipo de tarea.
- ## Enunciado: Descripción específica con todos los datos y condiciones.

Ejemplo:

Problema: Análisis y simplificación de una gramática.

Enunciado:

Dada la gramática GG:

$S \rightarrow AB/AC/BCS \rightarrow AB/AC/BC$

$A \rightarrow BC/aA \rightarrow BC/a$

$B \rightarrow AC/bB \rightarrow AC/b$

$C \rightarrow AB/cAC \rightarrow AB/cA$

Determine las propiedades del lenguaje generado por la gramática y realice un análisis de su estructura.

Es una buena práctica, aunque no necesario, añadir siempre en el enunciado todo lo que se desea que Fluxo **verifique explícitamente**, por ejemplo:

- Que la gramática esté en FNC antes de aplicar CYK.
- Que la simplificación ha eliminado todas las producciones inútiles.
- Que la conversión cumple las propiedades requeridas.
- Que se ha eliminado la recursividad directa e indirecta.

2. Instrucciones sintácticas y terminológicas

- Utilizar vocabulario técnico de manera rigurosa y coherente: Gramática Independiente del contexto (*GIC*), Gramática regular (*GR*), Autómata finito no determinista (*AFND*), etc.
- Ser preciso y específico en la descripción de pasos, condiciones y resultados.
- No introducir **ambigüedades**, por ejemplo:

Determinar si una cadena pertenece a un lenguaje usando CYK.

Siempre hay que comprobar si la gramática inicial está en FNC, pues no decir en ningún punto, ni en enunciado ni en solución, que la gramática ya se ha comprobado que está en FNC, ya que una instrucción de Fluxo es comprobar antes de aplicar el algoritmo CYK si la gramática inicial está en FNC, porque esto es una instrucción **ambigua**, le decimos que lo compruebe y a la vez que ya se ha comprobado, por tanto muchas veces no lo comprobará.

- **Verificación y control de calidad**
 - No omitir nunca el paso de verificación: si el ejercicio requiere comprobar propiedades, estas deben incluirse explícitamente en el enunciado y desarrollarse en la solución.
 - Evitar suposiciones no justificadas: nunca asumir que una gramática está en FNC, o que una propiedad se cumple, si no se ha verificado explícitamente.

2.1. Resumen de buenas prácticas

- Formular prompts completos, estructurados y específicos.
- Utilizar formato Markdown para toda la entrada y salida.
- Presentar tablas y símbolos matemáticos según los estándares definidos.
- Seguir la estructura de pasos de los algoritmos implementados.
- Incluir verificación exhaustiva en cada problema.
- Ser riguroso en la terminología y evitar ambigüedades.
- Añadir en el enunciado todas las verificaciones que se desean obtener.

3. Ejemplo de prompt óptimo para Fluxo

Problema: Eliminación de la recursividad por la izquierda en una gramática independiente de contexto (*GIC*)

Enunciado

Elimina la recursividad por la izquierda (directa e indirecta) de la siguiente gramática independiente de contexto (GIC):

- $S \rightarrow SaB \mid SBa \mid ab$

- $A \rightarrow Baa \mid Cbc$

- $B \rightarrow CA b \mid CbA \mid ac$

- $C \rightarrow Abb \mid SaaB \mid Sb \mid cc$

Solucion

Paso 1: Gramática original

$S \rightarrow SaB \mid SBa \mid ab$

$A \rightarrow Baa \mid Cbc$

$B \rightarrow CA b \mid CbA \mid ac$

$C \rightarrow Abb \mid SaaB \mid Sb \mid cc$

Paso 2: Detección de problemas

- Recursividad directa:

Regla de S: ' $S \rightarrow SaB \mid SBa$ '

- Recursividad indirecta:

Ciclo entre no terminales ($A \rightarrow B \rightarrow C \rightarrow S \rightarrow A$)

Paso 3: Procedimiento de eliminación

3.1 Ordenar no terminales

Jerarquía según dependencias:

' S (nivel 1) $\rightarrow A$ (nivel 2) $\rightarrow B$ (nivel 3) $\rightarrow C$ (nivel 4)'

3.2 Sustitución en cascada

1. Sustituir C en B:

$B \rightarrow AbbAb \mid SaaBA b \mid SbAb \mid ccAb$

$\mid AbbbA \mid SaaBbbA \mid SbbA \mid ccbA$

$\mid ac$

2. Sustituir B en A:

$A \rightarrow (CAb|CbA|ac)aa \mid Cbc$

3. Sustituir A en S:

(No requiere cambios adicionales)

3.3 Eliminar recursividad directa en S

Transformación estándar para producciones recursivas:

$S \rightarrow abS'$

$S' \rightarrow aBS' \mid BaS' \mid \varepsilon$

Paso 4: Gramática transformada final

$S \rightarrow abS'$

$S' \rightarrow aBS' \mid BaS' \mid \varepsilon$

$A \rightarrow Baa \mid Cbc$

$B \rightarrow CAb \mid CbA \mid ac$

$C \rightarrow Abb \mid SaaB \mid Sb \mid cc$

Paso 5: Verificación de resultados

- Recursividad directa eliminada:

Ninguna producción de S inicia con S

- Jerarquía preservada:

No hay ciclos entre no terminales

- Equivalencia garantizada:

Mismo lenguaje generado con estructura no recursiva

Se logra:

1. Eliminar la autorreferencia directa en `S`

2. Romper el ciclo indirecto $C \rightarrow SaaB \rightarrow B \rightarrow CAb \rightarrow C$

¿Por qué funciona sin reordenar no terminales?

1. La recursividad indirecta dependía de la producción $C \rightarrow SaaB$

2. Al modificar `S` para que nunca empiece con `S`, se evita que `C` pueda regenerar la recursividad.